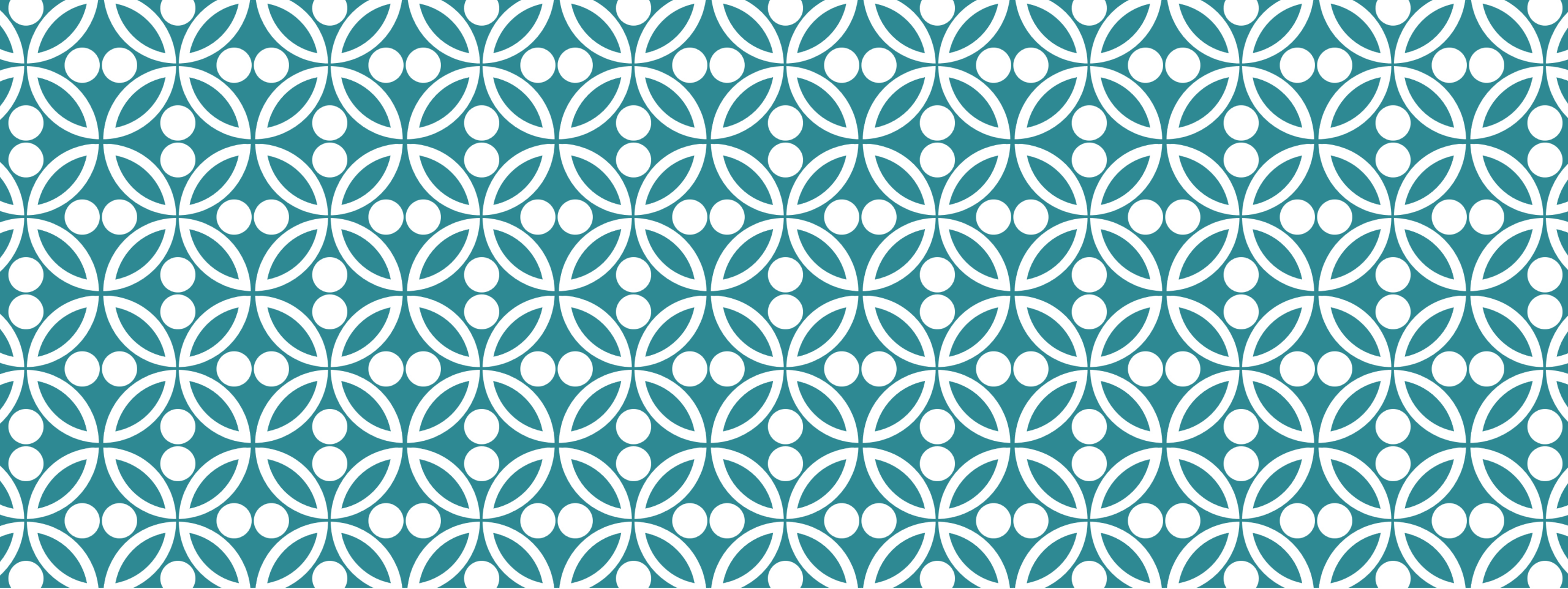




SESSION 1: RECAP R AND TIDYVERSE

DR. SOFIA GIL-CLAVEL

- ❖ Using R in RStudio
- ❖ System Library
- ❖ Tidyverse
- ❖ Data Handling
- ❖ Data Visualization

Looking for help

1. USING R IN RSTUDIO

What is R?
What is RStudio?

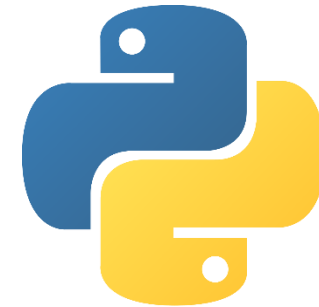
APPLICATION SOFTWARE

The main function of application software is to help and assist a user to perform specific tasks in a computer. Application programs can be developed with different software languages and tools.

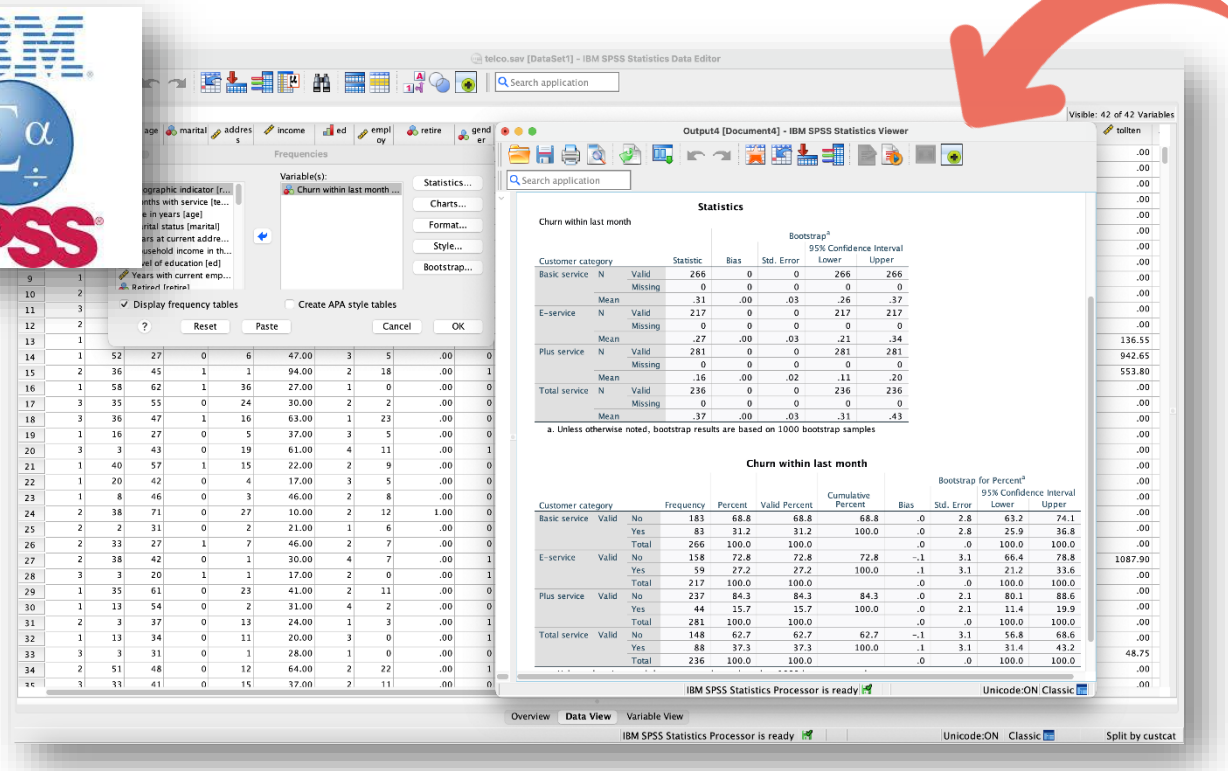


PROGRAMMING LANGUAGES

Programming languages are used to write programs that allow user/machine communication.

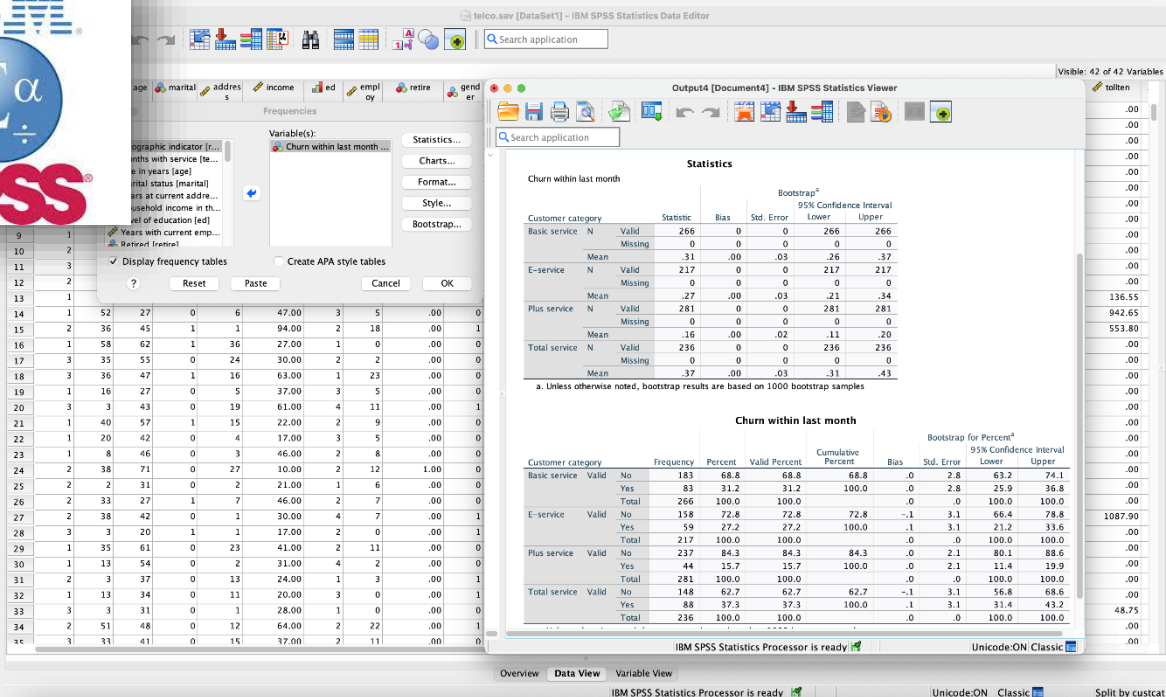


APPLICATION SOFTWARE VS. PROGRAMMING LANGUAGE



If you want to do something, then you just need to “click click click”

APPLICATION SOFTWARE VS. PROGRAMMING LANGUAGE



```

Packages Windows Help

(2024-10-31 ucrt) -- "File of Leaves"
Copyright 2024 The R Foundation for Statistical Computing
Platform: x86_64-w64-mingw32/x64

R is free software and comes with ABSOLUTELY NO WARRANTY.
You are welcome to redistribute it under certain conditions.
Type 'license()' or 'licence()' for distribution details.

R is a collaborative project with many contributors.
Type 'contributors()' for more information and
'citation()' on how to cite R or R packages in publications.

Type 'demo()' for some demos, 'help()' for on-line help, or
'help.start()' for an HTML browser interface to help.
Type 'q()' to quit R.

> |
```

If you want to do something, then you just need to write code, “code code code”

THE LOGIC OF



THE LOGIC OF



➤ Data is stored in variables.



```
> 2
[1] 2
> "hello"
[1] "hello"
~
```



THE LOGIC OF



- Data is stored in variables.
 - This means that we always need to assign numbers, strings, and databases to variables.
 - A variable is a string of characters. It is assigned a value by using the special characters “=” or “<-”.



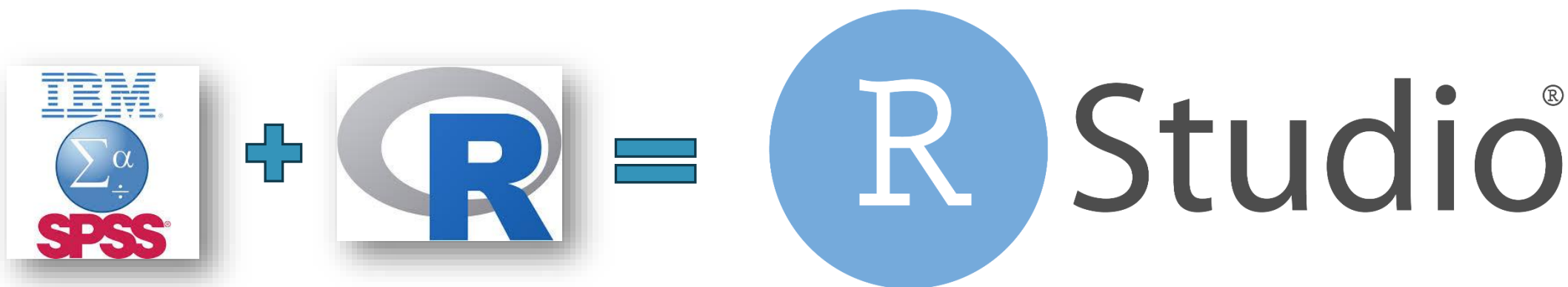
```
> 2
[1] 2
> "hello"
[1] "hello"
>
> x=2
> y<-"hello"
>
```

THE LOGIC OF



- Data is stored in variables.
 - This means that we always need to assign numbers, strings, and databases to variables.
 - A variable is a string of characters. It is assigned a value by using the special characters “=” or “<-”.
- Variables are case sensitive, i.e. “HELLO” is not the same as “hello” or “Hello”. So, always try to remember the exact characters you used.

```
> 2
[1] 2
> "hello"
[1] "hello"
>
> x=2
> y<-"hello"
>
> HELLO
Error: object 'HELLO' not found
>
> HELLO="hello"
> hello<-"no way"
>
> HELLO
[1] "hello"
>
> hello
[1] "no way"
>
```





```

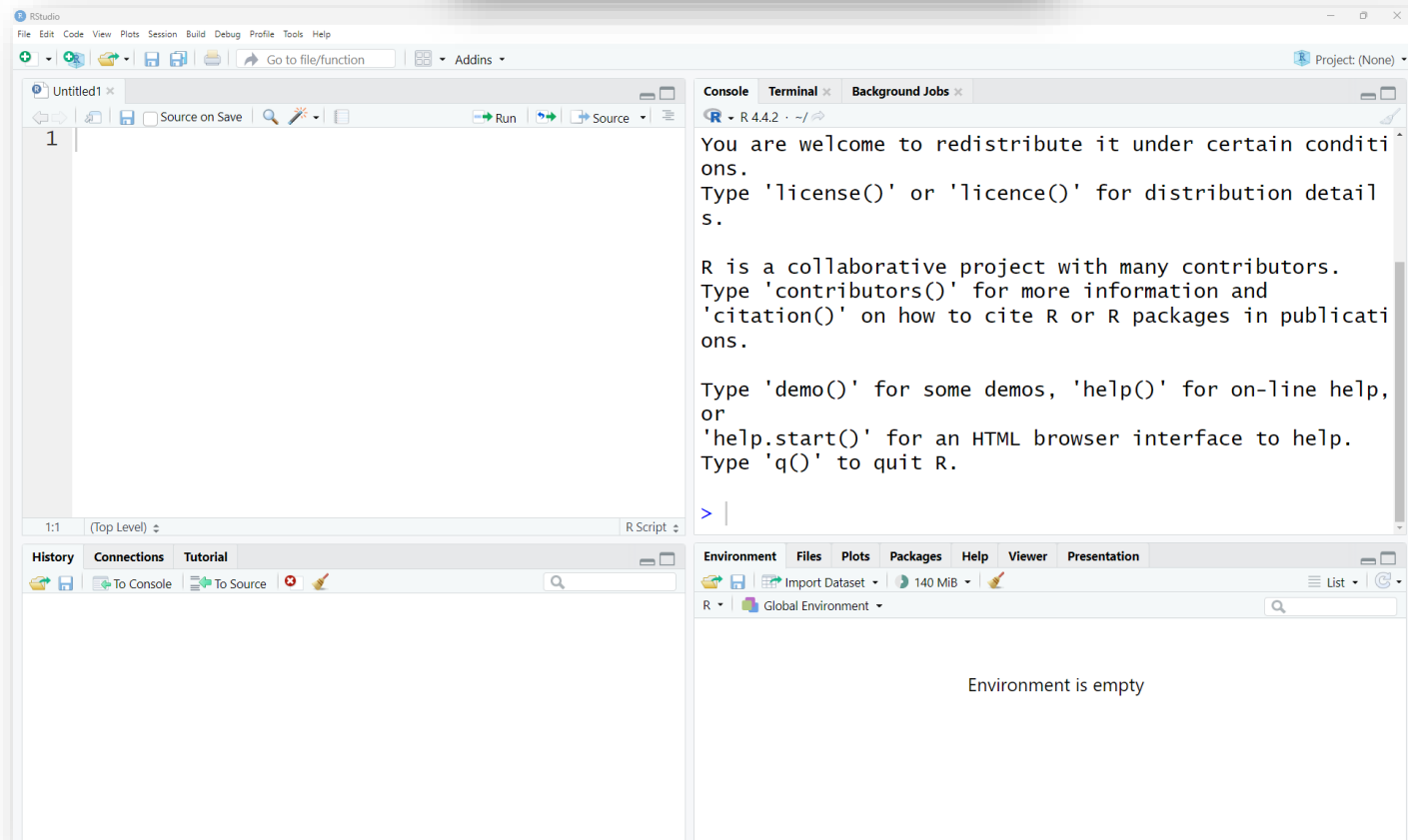
(2024-10-31 ucrt) -- "File of Leaves"
Copyright (c) 2024 The R Foundation for Statistical Computing
Platform: x86_64-w64-mingw32/x64

R is free software and comes with ABSOLUTELY NO WARRANTY.
You are welcome to redistribute it under certain conditions.
Type 'license()' or 'licence()' for distribution details.

R is a collaborative project with many contributors.
Type 'contributors()' for more information and
'citation()' on how to cite R or R packages in publications.

Type 'demo()' for some demos, 'help()' for on-line help, or
'help.start()' for an HTML browser interface to help.
Type 'q()' to quit R.

> |
```



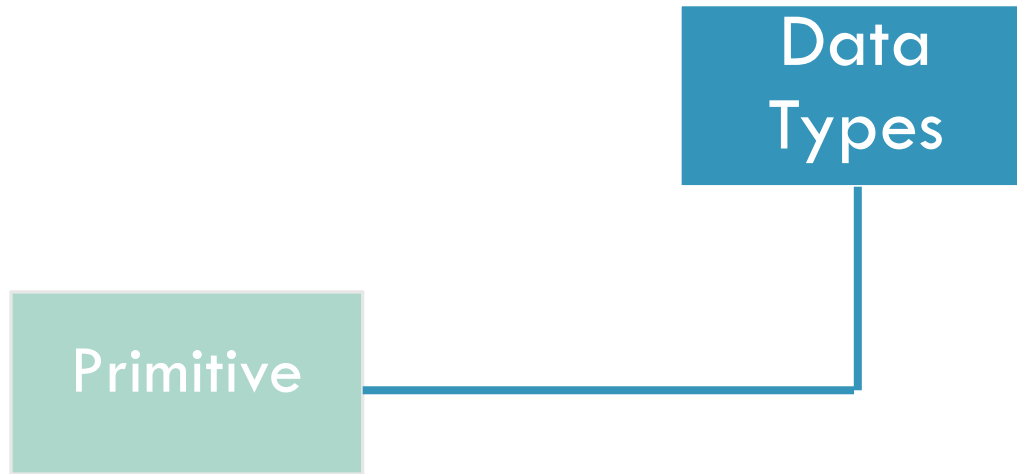
Now you can “click click click” to access certain functions, but you still need to “code code code” to tell the computer what to do.

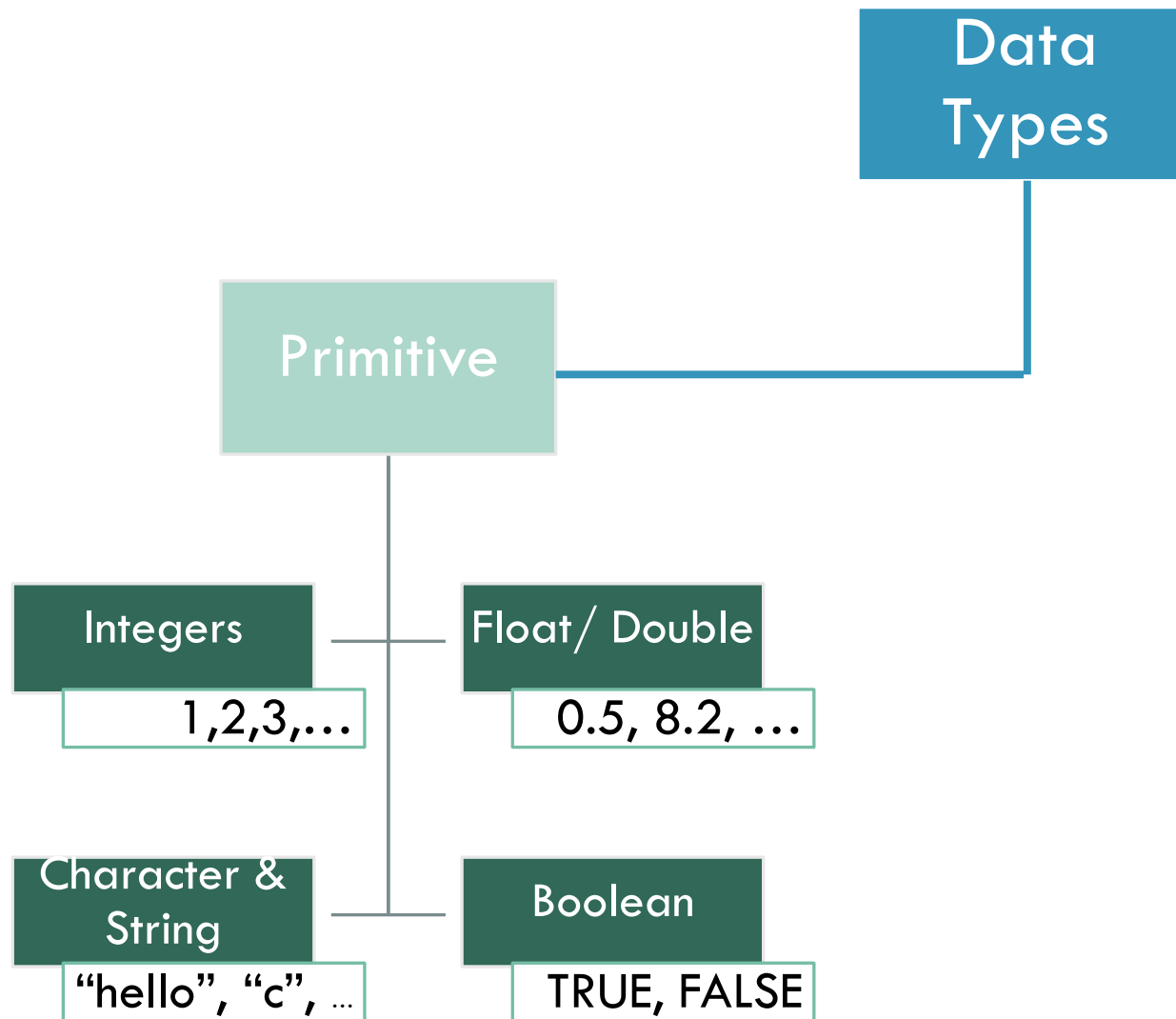
LET'S OPEN RSTUDIO!



2. SYSTEM LIBRARY

Data Types





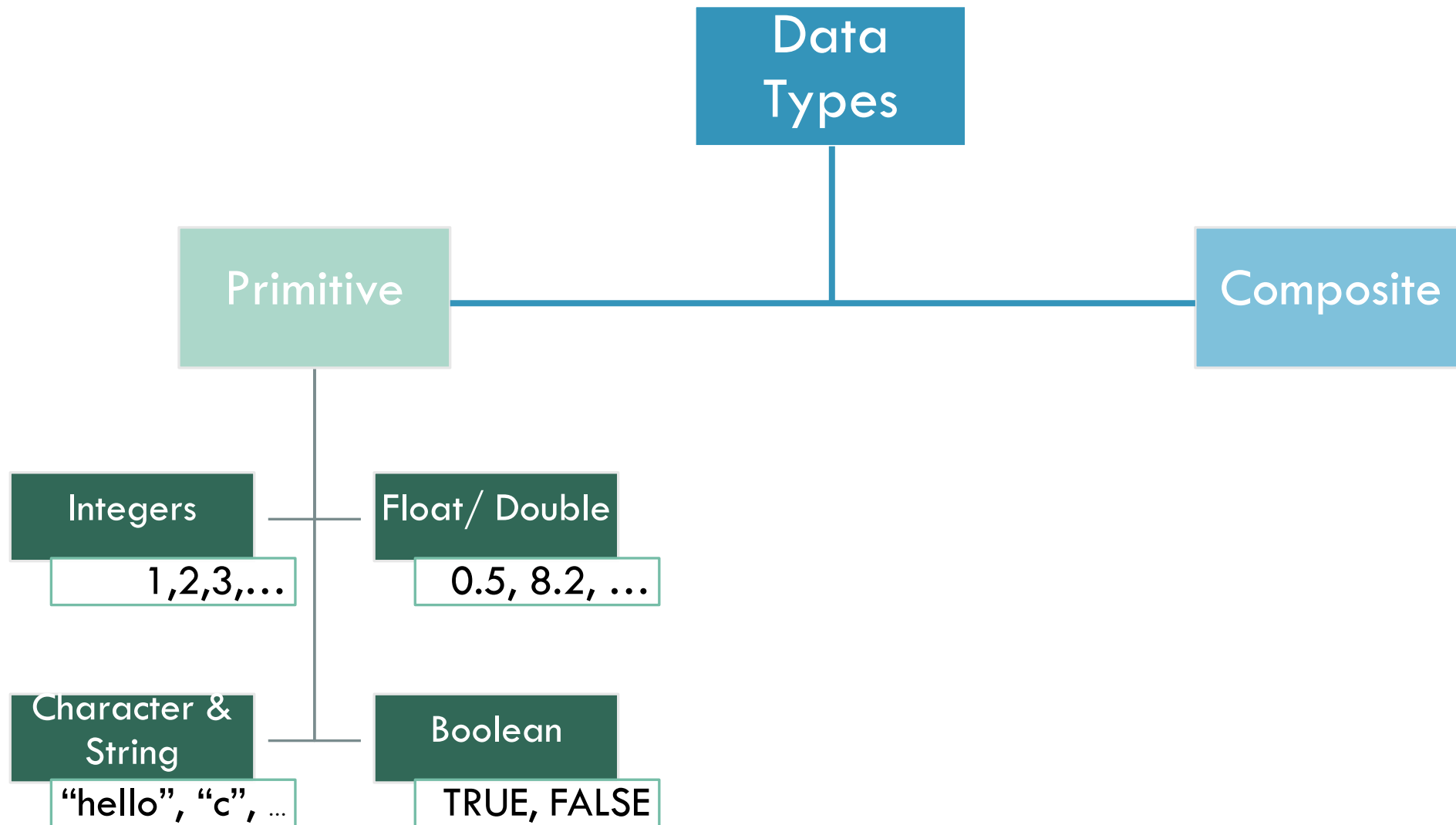
PRIMITIVE DATA AND ITS OPERATORS

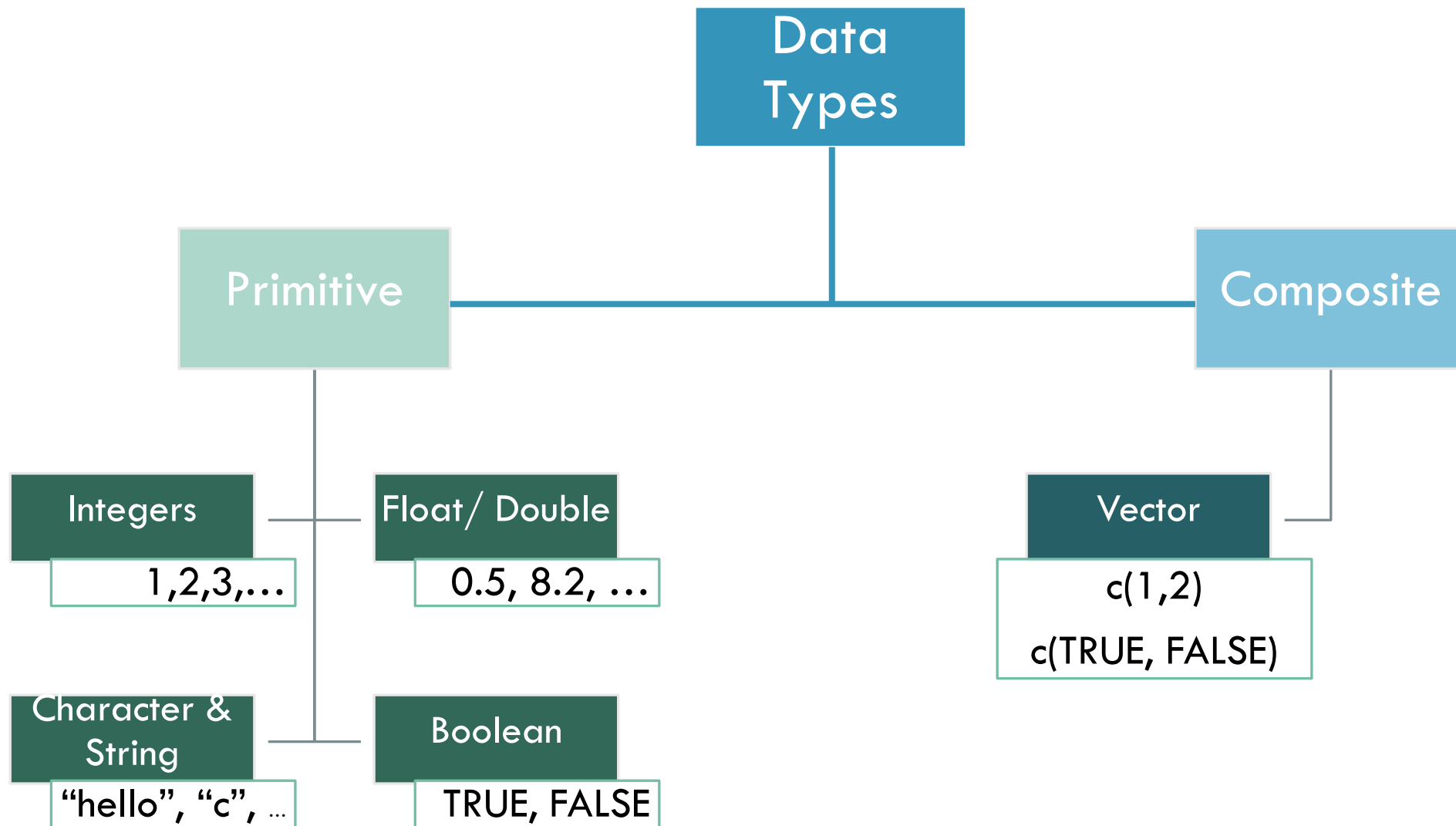
Operators					
Arithmetic		Comparative		Logic	
+	addition	<	less than	! x	logic NO
-	subtraction	>	more than	x & y	element-wise AND
*	multiplication	<=	less or equal than	x && y	single comparison AND
/	division	>=	more or equal than	x y	element-wise OR
^	power	==	equal than	x y	single comparison OR
%%	integer division	!=	different than	xor(x,y)	exclusive OR

USING THE PRIMITIVE DATA AND THEIR OPERATORS

Solve the following:

- The addition of two and four.
- Is one third bigger than one fourth?





SOME USEFUL FUNCTIONS

For string vectors:

- `chartr()`
- `tolower()`
- `toupper()`
- `toString()`
- `nchar()`
- `nzchar()`
- `noquote()`
- `pmatch()`
- `charmatch()`
- `match()`
- `char.expand()`
- `grep()`
- `grepl()`
- `sub()`
- `substr()`
- `substring()`
- `strsplit()`
- `strtrim()`

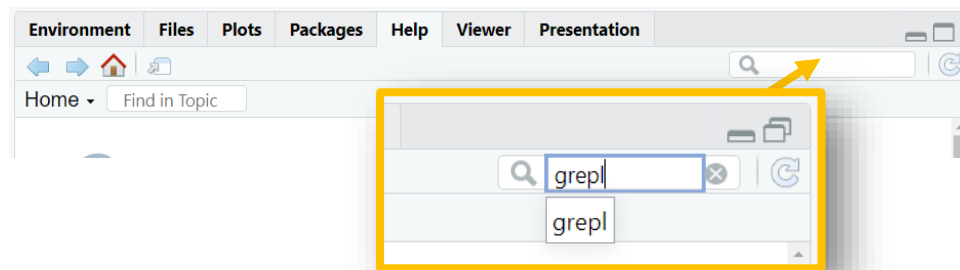
For integer and double vectors:

<code>sum(x)</code>	<code>round(x, n)</code>
<code>prod(x)</code>	<code>rev(x)</code>
<code>max(x)</code>	<code>sort(x)</code>
<code>min(x)</code>	
<code>which.max(x)</code>	<code>rank(x)</code>
<code>which.min(x)</code>	<code>log(x, base)</code>
<code>range(x)</code>	<code>scale(x)</code>
<code>length(x)</code>	
<code>mean(x)</code>	<code>pmin(x, y, ...)</code>
<code>median(x)</code>	<code>pmax(x, y, ...)</code>
<code>var(x) o cov(x)</code>	<code>cumsum(x)</code>
<code>cor(x)</code>	<code>cumprod(x)</code>
<code>var(x, y) o cov(x, y)</code>	<code>cummin(x)</code>
<code>cor(x, y)</code>	<code>cummax(x)</code>

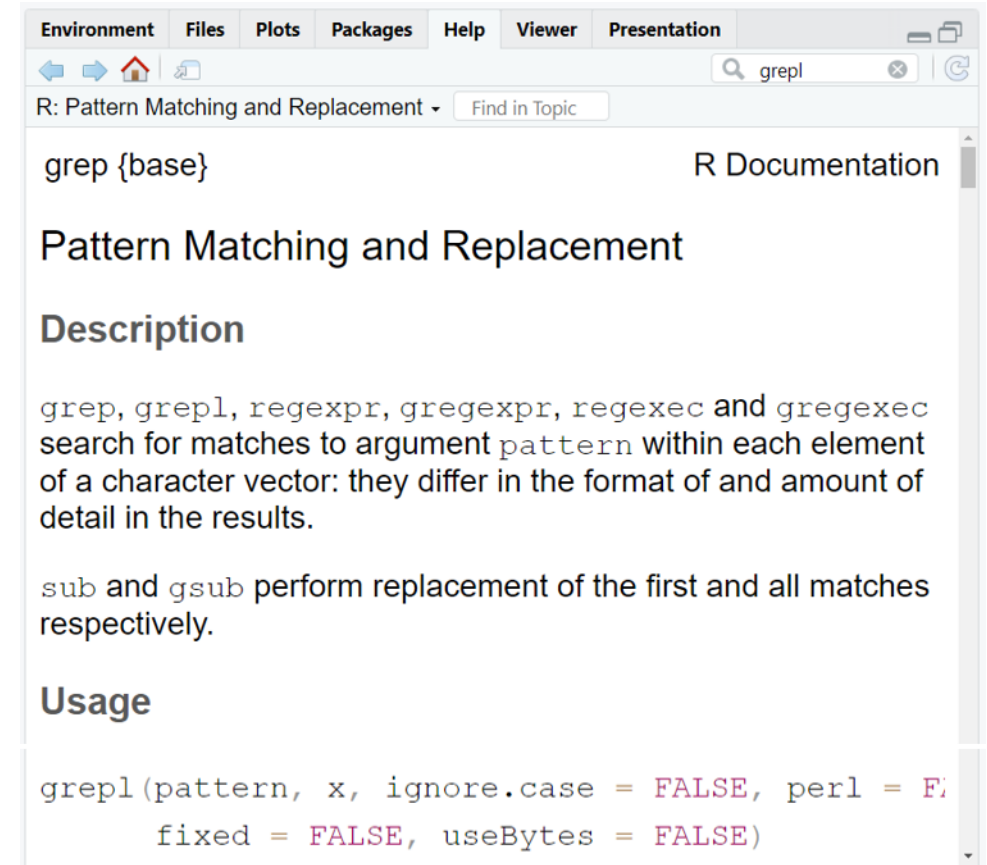
LOOKING FOR HELP 1: FUNCTIONS

If you want to learn more about a function, R offers a couple of ways to do so.

1. Go to the “Help” panel.
2. Look for the name of the function.
Look for “grepl” and click on it.



3. This will open a file with an explanation of the function and how to use it.



4. If you scroll all the way down, you will find some examples on how to use it.

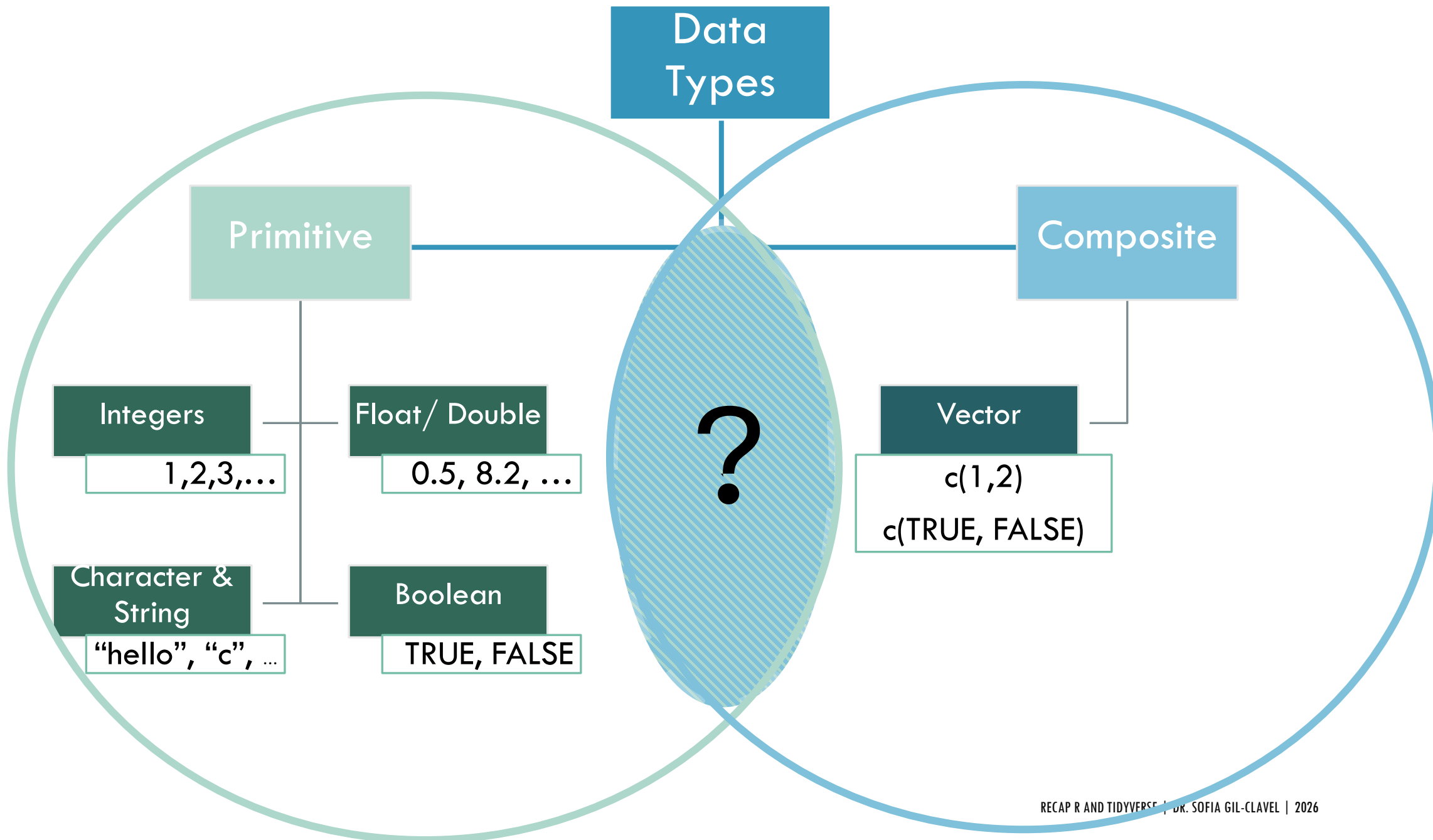
USING VECTORS AND THEIR FUNCTIONS

Build the following vectors:

- Vector called ONE with the numbers from 0 to 5.
- Vector called TWO with the numbers from -5 to 0.
- Vector called THREE with the following strings “banana”, “Manhattan”, “Brazilian”.

Obtain the following:

- The mean of ONE.
- The correlation between ONE and TWO.
- Turn all the values in THREE to lowercase and save the result in THREE.
- A Boolean vector that tells whether the values in THREE contain the string “an”.



Data Types

Primitive

Integers

1,2,3,...

Float/ Double

0.5, 8.2, ...

Character & String

"hello", "c", ...

Boolean

TRUE, FALSE

Data Frame

```
n = c(2, 3, 5)
s = c("aa", "bb", "cc")
b = c(TRUE, FALSE, TRUE)
df = data.frame(n, s, b)
```

Composite

Vector

c(1,2)
c(TRUE, FALSE)



DATAFRAME FUNCTIONS

The dataframe name

The columns:
`names(mtcars)`
or
`colnames(mtcars)`

The row names:
`row.names(mtcars)`

```
> mtcars
```

	mpg	cyl	disp	hp	drat	wt	...
Mazda RX4	21.0	6	160	110	3.90	2.62	...
Mazda RX4 Wag	21.0	6	160	110	3.90	2.88	...
Datsun 710	22.8	4	108	93	3.85	2.32	...
.....							

A variable:
`mtcars$disp`

A value or element:
`mtcars$wt[3]`

USING DATAFRAMES

Using base R, build a dataframe named DT with the following columns:

- `NAME=c("Susana","Manuel","Orlando")`
- `AGE=c(23,43,65)`
- `SEX=c("female", "male", "female")`

Using the dataframe functions and primitive data operators, add an extra column to your dataframe to indicate whether a person is a woman over 65.

3. TIDYVERSE

**BUT WHAT IF
BASE R DOES NOT
HAVE THE
FUNCTION THAT
YOU NEED?**

Then you need to
download a
package!



WHAT IS A PACKAGE?

- R packages are extensions to the R statistical programming language.
- R packages contain code, data, and documentation in a standardized collection format that can be installed by users of R, typically via a centralized software repository such as CRAN (the Comprehensive R Archive Network).



[CRAN](#)
[Mirrors](#)
[What's new?](#)
[Search](#)
[CRAN Team](#)

[About R](#)
[R Homepage](#)
[The R Journal](#)

[Software](#)
[R Sources](#)
[R Binaries](#)
[Packages](#)
[Task Views](#)
[Other](#)

[Documentation](#)
[Manuals](#)
[FAQs](#)
[Contributed](#)

[Donations](#)
[Donate](#)

The Comprehensive R Archive Network

Download and Install R

Precompiled binary distributions of the base system and contributed packages, **Windows and Mac** users most likely want one of these versions of R:

- [Download R for Linux \(Debian, Fedora/Redhat, Ubuntu\)](#)
- [Download R for macOS](#)
- [Download R for Windows](#)

R is part of many Linux distributions, you should check with your Linux package management system in addition to the link above.

Source Code for all Platforms

Windows and Mac users most likely want to download the precompiled binaries listed in the upper box, not the source code. The sources have to be compiled before you can use them. If you do not know what this means, you probably do not want to do it!

- The latest release (2023-10-31, Eye Holes) [R-4.3.2.tar.gz](#), read [what's new](#) in the latest version.
- Sources of [R alpha and beta releases](#) (daily snapshots, created only in time periods before a planned release).
- Daily snapshots of current patched and development versions are [available here](#). Please read about [new features and bug fixes](#) before filing corresponding feature requests or bug reports.
- Source code of older versions of R is [available here](#).
- Contributed extension [packages](#)

Questions About R

- If you have questions about R like how to download and install the software, or what the license terms are, please read our [answers to frequently asked questions](#) before you send an email.

Supporting CRAN

- CRAN operations, most importantly hosting, checking, distributing, and archiving of R add-on packages for various platforms, crucially rely on technical, emotional, and financial support by the R community.

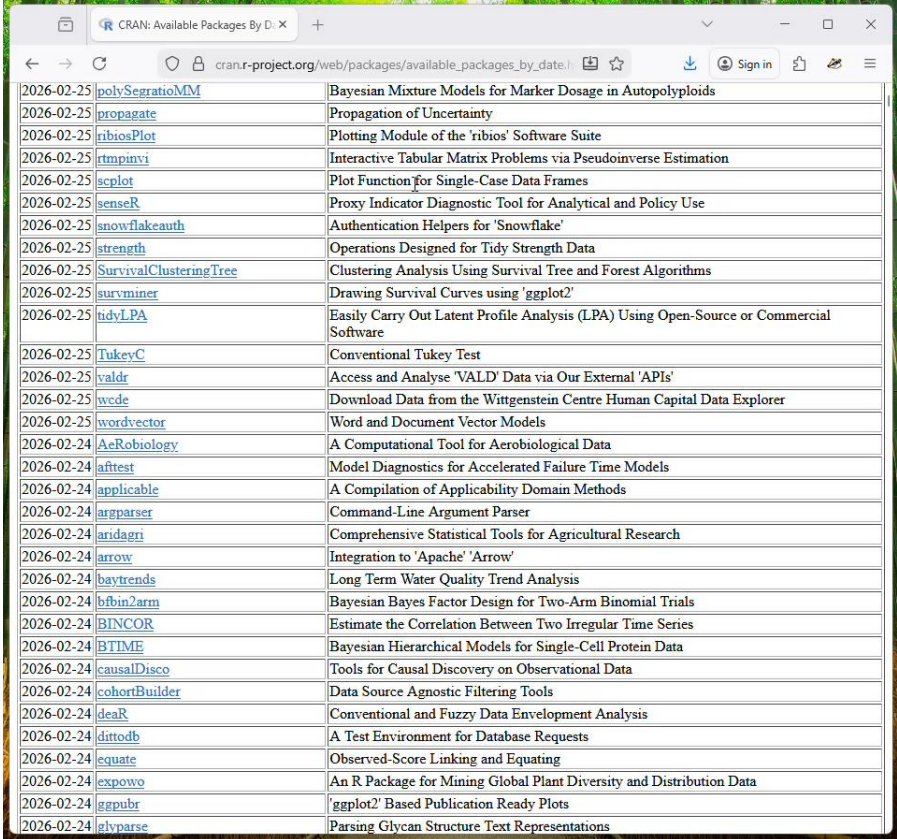
Please consider making [financial contributions](#) to the R Foundation for Statistical Computing.

<https://cran.r-project.org/>

HOW MANY PACKAGES ARE OUT THERE?

- There are thousands of packages!
- This makes it hard to keep track of the best ones or the most reliable ones!

Therefore, we need to be careful when using R-packages. As plenty of them, may not be the best for our goal.



The screenshot shows a web browser window displaying the CRAN website's 'Available Packages By Date' page. The browser's address bar shows the URL: https://cran.r-project.org/web/packages/available_packages_by_date.html. The page content is a table with three columns: the date of the update, the package name, and a brief description of the package. The table lists various R packages, including polySegratioMM, propagate, ribosPlot, rtmapinv, scplot, senseR, snowflakeauth, strength, SurvivalClusteringTree, survminer, tidyLPA, TukeyC, valdr, weede, wordvector, AeRobiology, afitest, applicable, argparser, aridagri, arrow, baytrends, bfbn2arm, BINCOR, BTIME, causalDisco, cohortBuilder, deaR, dittodb, equate, expowo, ggpubr, and glvparse. Each row represents a package update, with the date in the first column, the package name in the second, and the description in the third.

Date	Package Name	Description
2026-02-25	polySegratioMM	Bayesian Mixture Models for Marker Dosage in Autopolyploids
2026-02-25	propagate	Propagation of Uncertainty
2026-02-25	ribosPlot	Plotting Module of the 'ribios' Software Suite
2026-02-25	rtmapinv	Interactive Tabular Matrix Problems via Pseudoinverse Estimation
2026-02-25	scplot	Plot Function for Single-Case Data Frames
2026-02-25	senseR	Proxy Indicator Diagnostic Tool for Analytical and Policy Use
2026-02-25	snowflakeauth	Authentication Helpers for 'Snowflake'
2026-02-25	strength	Operations Designed for Tidy Strength Data
2026-02-25	SurvivalClusteringTree	Clustering Analysis Using Survival Tree and Forest Algorithms
2026-02-25	survminer	Drawing Survival Curves using 'ggplot2'
2026-02-25	tidyLPA	Easily Carry Out Latent Profile Analysis (LPA) Using Open-Source or Commercial Software
2026-02-25	TukeyC	Conventional Tukey Test
2026-02-25	valdr	Access and Analyse 'VALD' Data via Our External 'APIs'
2026-02-25	weede	Download Data from the Wittgenstein Centre Human Capital Data Explorer
2026-02-25	wordvector	Word and Document Vector Models
2026-02-24	AeRobiology	A Computational Tool for Aerobiological Data
2026-02-24	afitest	Model Diagnostics for Accelerated Failure Time Models
2026-02-24	applicable	A Compilation of Applicability Domain Methods
2026-02-24	argparser	Command-Line Argument Parser
2026-02-24	aridagri	Comprehensive Statistical Tools for Agricultural Research
2026-02-24	arrow	Integration to 'Apache' 'Arrow'
2026-02-24	baytrends	Long Term Water Quality Trend Analysis
2026-02-24	bfbn2arm	Bayesian Bayes Factor Design for Two-Arm Binomial Trials
2026-02-24	BINCOR	Estimate the Correlation Between Two Irregular Time Series
2026-02-24	BTIME	Bayesian Hierarchical Models for Single-Cell Protein Data
2026-02-24	causalDisco	Tools for Causal Discovery on Observational Data
2026-02-24	cohortBuilder	Data Source Agnostic Filtering Tools
2026-02-24	deaR	Conventional and Fuzzy Data Envelopment Analysis
2026-02-24	dittodb	A Test Environment for Database Requests
2026-02-24	equate	Observed-Score Linking and Equating
2026-02-24	expowo	An R Package for Mining Global Plant Diversity and Distribution Data
2026-02-24	ggpubr	'ggplot2' Based Publication Ready Plots
2026-02-24	glvparse	Parsing Glycan Structure Text Representations

https://cran.r-project.org/web/packages/available_packages_by_date.html

❖ TIDYVERSE: [HTTPS://WWW.TIDYVERSE.ORG/](https://www.tidyverse.org/)



R packages for data science

The tidyverse is an opinionated **collection of R packages** designed for data science. All packages share an underlying design philosophy, grammar, and data structures.

Install the complete tidyverse with:

```
install.packages("tidyverse")
```

LOADING A PACKAGE

- Optional: Install the package tidyverse.
- Load the package tidyverse.

5 MINS BREAK



THE TIBBLES



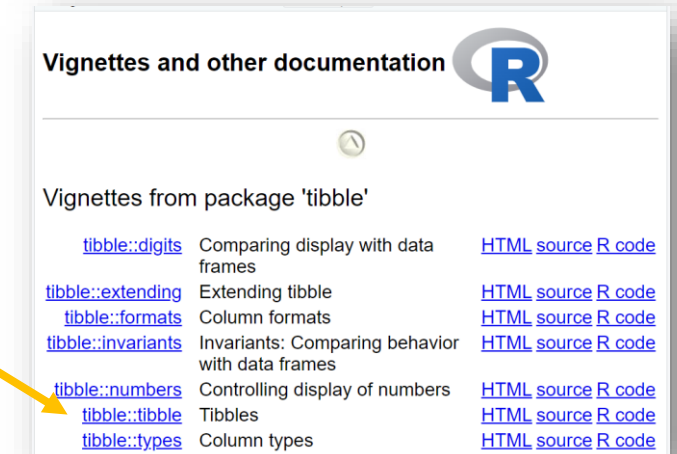
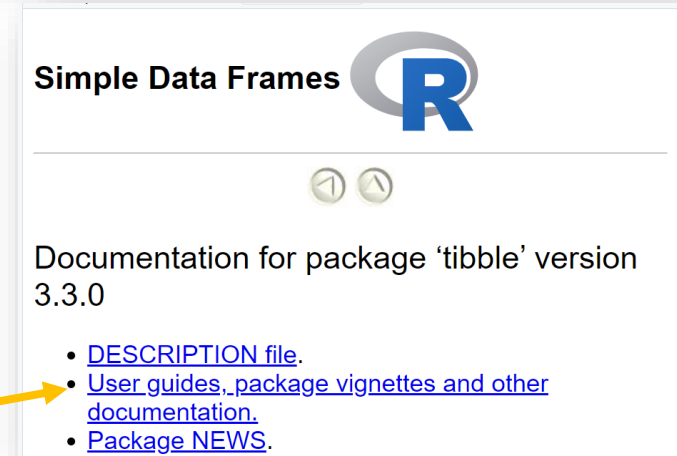
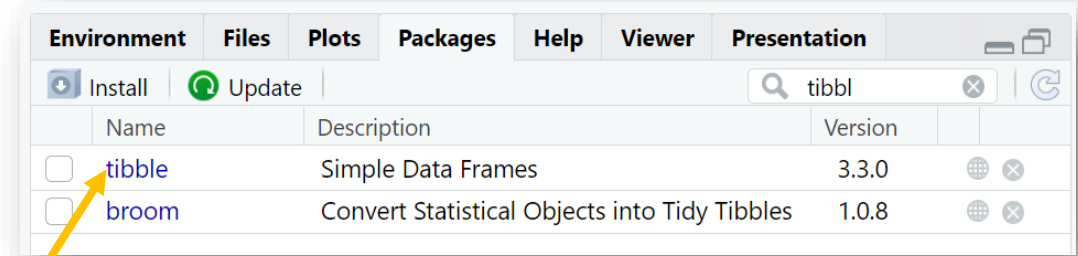
A **tibble**, or `tbl_df`, is a modern reimagining of the `data.frame`, keeping what time has proven to be effective, and throwing out what is not. Tibbles are `data.frames` that are lazy and surly: they do less (i.e. they don't change variable names or types, and don't do partial matching) and complain more (e.g. when a variable does not exist). This forces you to confront problems earlier, typically leading to cleaner, more expressive code. Tibbles also have an enhanced `print()` method which makes them easier to use with large datasets containing complex objects.

<https://tibble.tidyverse.org/>

LOOKING FOR HELP 2: VIGNETTES

If you want to learn more about a package, R offers a couple of ways to do so.

1. Go to the “Packages” panel.
2. Look for the package “tibble” and click on it.
3. This will open a file with the links to some tutorials, and the links to all the functions and their help pages.
4. Click on “User guides, package vignettes and other documentation.”
5. From there, press on “`tibble::tibble`”.



USING TIBBLES

Using tibble, rebuild the DT dataframe. Remember, it has the following columns:

- `NAME=c("Susana","Manuel","Orlando")`
- `AGE=c(23,43,65)`
- `SEX=c("female", "male", "female")`

How would you add an extra column to your tibble to indicate whether a person is a woman over 65?

THE PIPE FUNCTION



Basic piping

- `x %>% f` is equivalent to `f(x)`
- `x %>% f(y)` is equivalent to `f(x, y)`
- `x %>% f %>% g %>% h` is equivalent to `h(g(f(x)))`

Here, “equivalent” is not technically exact: evaluation is non-standard, and the left-hand side is evaluated before passed on to the right-hand side expression. However, in most cases this has no practical implication.

<https://magrittr.tidyverse.org/reference/pipe.html>

USING PIPES

Rewrite the following code using pipes:

- `mean(c(1,5,3,4,2,6,7))`

Hint:

`x %>% f` is equivalent to `f(x)`

- `cor(c(1,5,3,4,2,6,7),c(1,2,3,4,5,6,7))`

Hint:

`x %>% f(y)` is equivalent to `f(x, y)`

- `cor(cumsum(c(1,5,3,4,2,6,7)),c(1,2,3,4,5,6,7))`

Hint:

`x %>% f %>% g %>% h` is equivalent to `h(g(f(x)))`

4. DATA HANDLING

LET'S INTRODUCE THE DATA!

Today, we will use the database “**ChickWeight**” from base R.

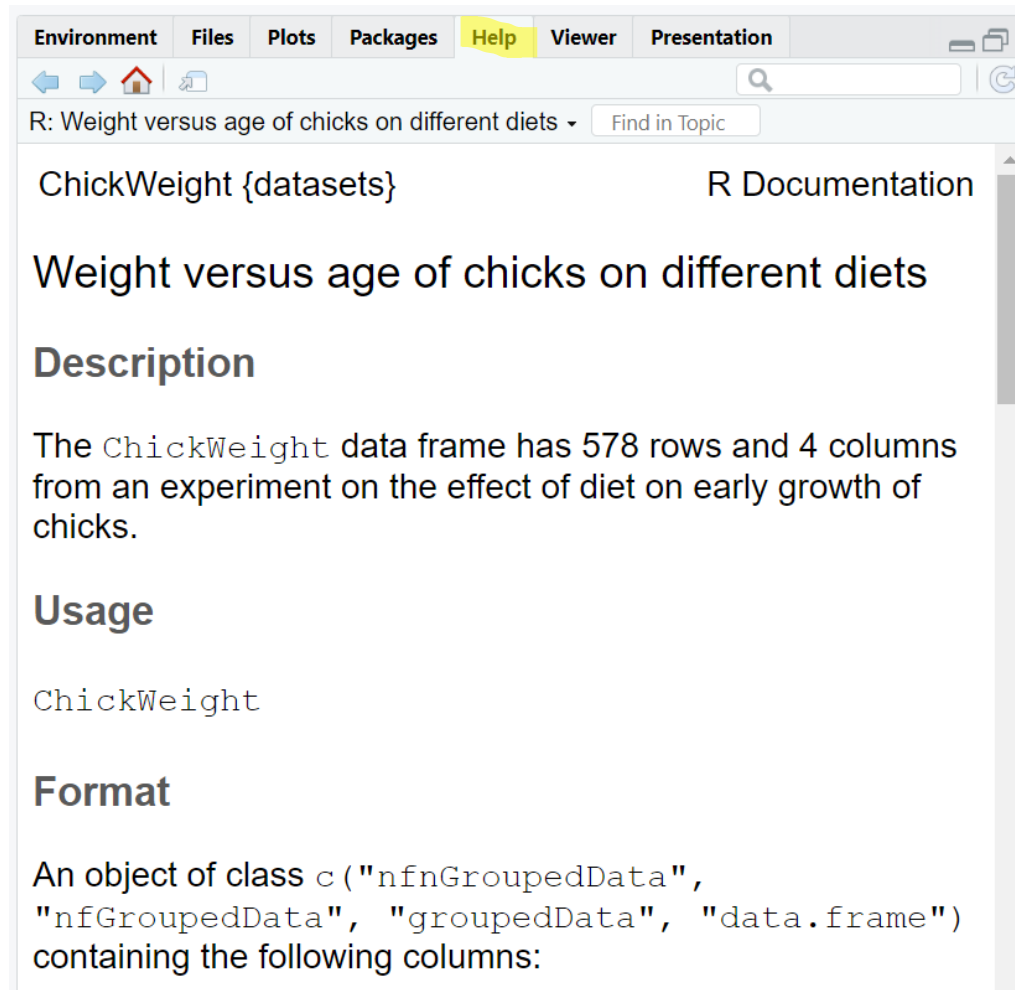
To open it, you just need to write down the name in the console:

```
ChickWeight
```

To see in an independent window, you can use:

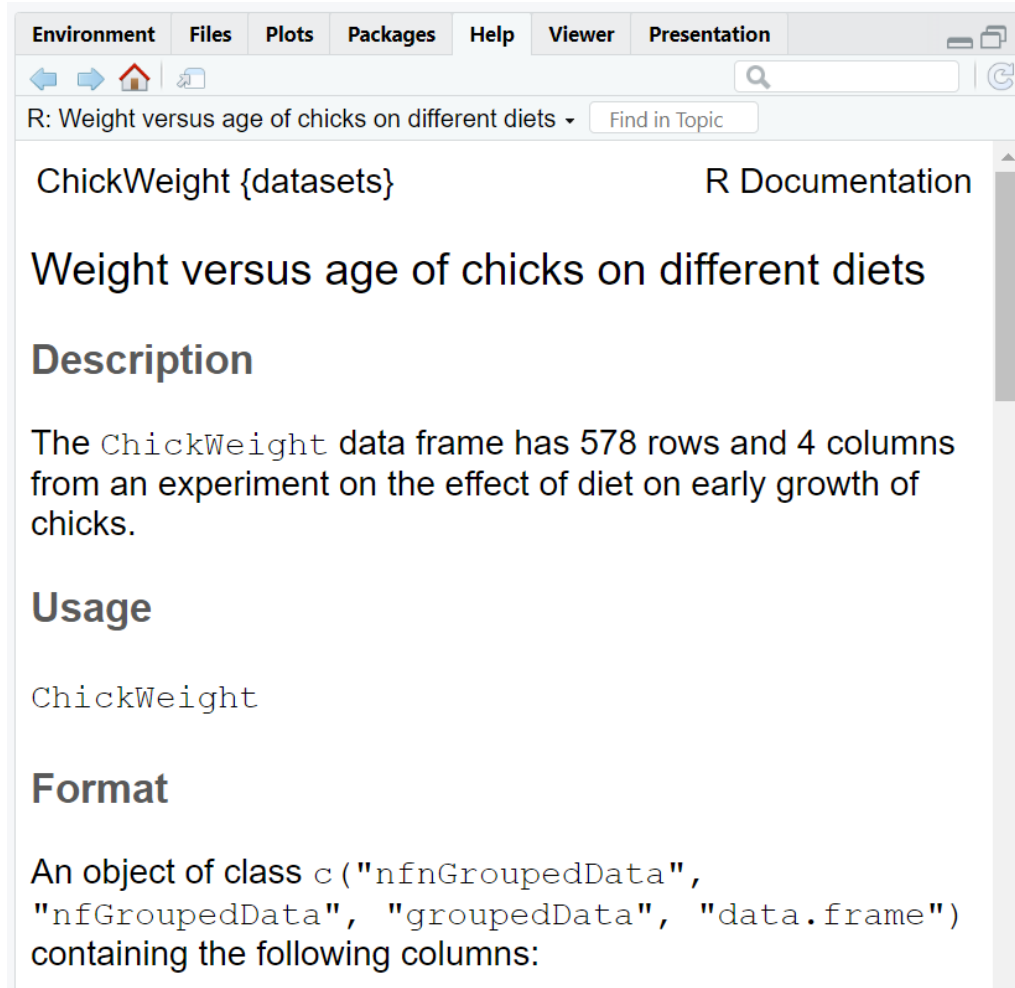
```
view(ChickWeight)
```

CHECKING THE VARIABLES



To learn what each variable represents, you can check its documentation in the “help” window.

CHECKING THE VARIABLES



Environment Files Plots Packages Help Viewer Presentation

R: Weight versus age of chicks on different diets Find in Topic

ChickWeight {datasets} R Documentation

Weight versus age of chicks on different diets

Description

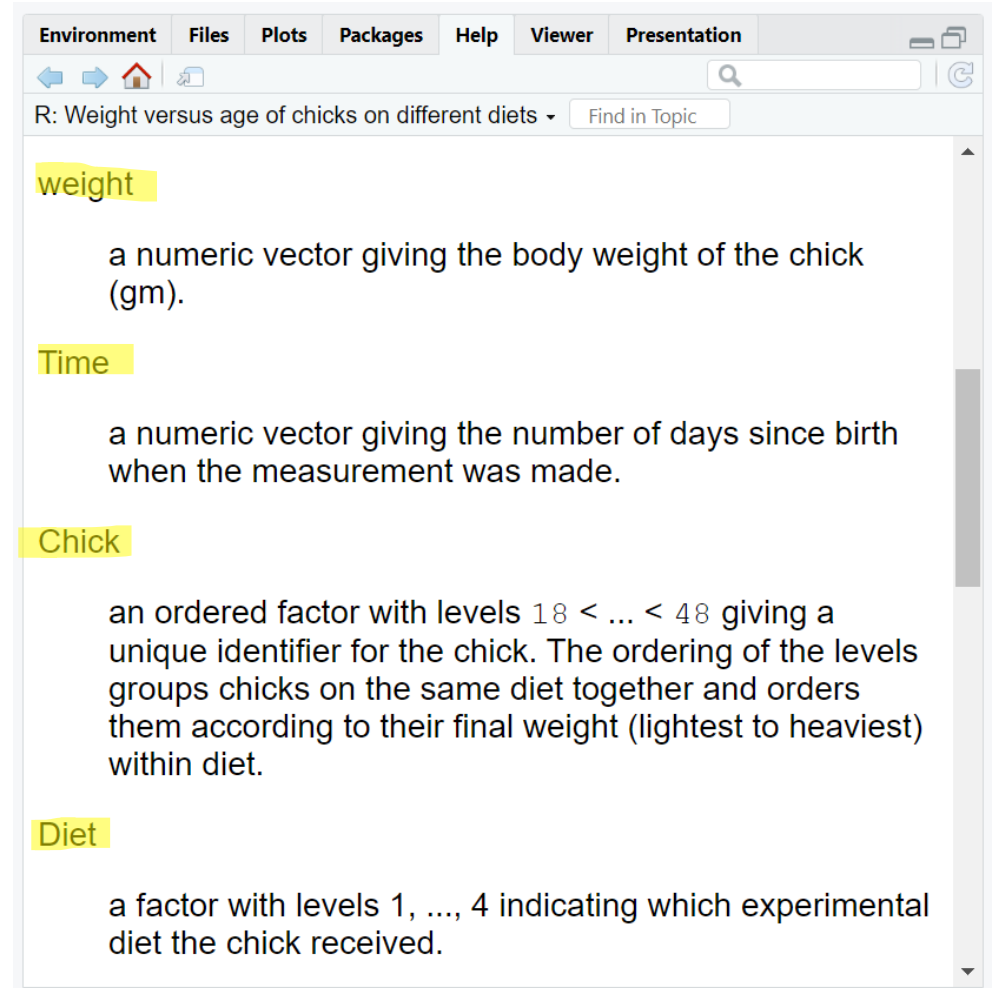
The `ChickWeight` data frame has 578 rows and 4 columns from an experiment on the effect of diet on early growth of chicks.

Usage

```
ChickWeight
```

Format

An object of class `c("nfnGroupedData", "nfGroupedData", "groupedData", "data.frame")` containing the following columns:



Environment Files Plots Packages Help Viewer Presentation

R: Weight versus age of chicks on different diets Find in Topic

weight

a numeric vector giving the body weight of the chick (gm).

Time

a numeric vector giving the number of days since birth when the measurement was made.

Chick

an ordered factor with levels `18 < ... < 48` giving a unique identifier for the chick. The ordering of the levels groups chicks on the same diet together and orders them according to their final weight (lightest to heaviest) within diet.

Diet

a factor with levels `1, ..., 4` indicating which experimental diet the chick received.

THE PACKAGE TO HANDLE DATA



<https://dplyr.tidyverse.org/>

dplyr is a grammar of data manipulation, providing a consistent set of verbs that help you solve the most common data manipulation challenges:

- `mutate()` adds new variables that are functions of existing variables
- `select()` picks variables based on their names.
- `filter()` picks cases based on their values.
- `summarise()` reduces multiple values down to a single summary.
- `arrange()` changes the ordering of the rows.

These all combine naturally with `group_by()` which allows you to perform any operation “by group”. You can learn more about them in `vignette("dplyr")`. As well as these single-table verbs, dplyr also provides a variety of two-table verbs, which you can learn about in `vignette("two-table")`.

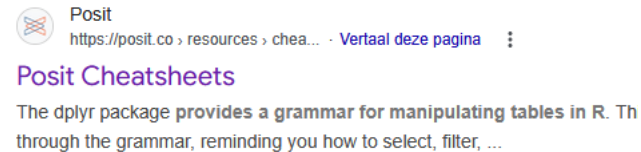
If you are new to dplyr, the best place to start is the [data transformation chapter](#) in R for Data Science.

LOOKING FOR HELP 3: CHEATSHEETS

All the “tidyverse” packages come with a Cheatsheet.

A Cheatsheet is an infographic with graphic explanations of what the package does and how the functions work.

1. Open your favorite browser (Google, Mozilla, Edge, etc.).
2. In the search section, type the words: “Posit”, which is the name of company behind tidyverse together with “Cheatsheet”.



3. In there, you will see Cheatsheets for different packages.

CHEATSHEET |

Data transformation with dplyr cheatsheet

The [dplyr](#) package provides a grammar for manipulating tables in R. This cheatsheet will guide you through the grammar,...

[VIEW THE CHEATSHEET](#)

4. Scroll down and find the one for “Dplyr”.

Data transformation with dplyr :: CHEATSHEET



dplyr functions work with pipes and expect tidy data. In tidy data:



Each **variable** is in its own **column**



Each **observation**, or **case**, is in its own **row**

pipes

$x \gg f(y)$ becomes $f(x, y)$

Summarize Cases

Apply **summary functions** to columns to create a new table of summary statistics. Summary functions take vectors as input and return one value (see back).

summary function



summarize(.data, ...)
Compute table of summaries.
mtcars > summarize(avg = mean(mpg))



count(.data, ..., wt = NULL, sort = FALSE, name = NULL) Count number of rows in each group defined by the variables in ... Also **tally()**, **add_count()**, **add_tally()**.
mtcars > count(cyl)

Group Cases

Use **group_by(.data, ..., add = FALSE, .drop = TRUE)** to create a "grouped" copy of a table grouped by columns in ... dplyr functions will manipulate each "group" separately and combine the results.



mtcars > group_by(cyl) > summarize(avg = mean(mpg))

Alternate grouping syntax with **.by** as an argument:

mtcars > summarize(avg = mean(mpg), .by = cyl)

Use **rowwise(.data, ...)** to group data into individual rows. dplyr functions will compute results for each row. Also apply functions to list-columns. See tidy cheat sheet for list-column workflow.



starwars > rowwise() > mutate(film_count = length(films))

ungroup(x, ...) Returns ungrouped copy of table.
g_mtcars <- mtcars > group_by(cyl)
ungroup(g_mtcars)

Manipulate Cases

EXTRACT CASES

Row functions return a subset of rows as a new table.



filter(.data, ..., .preserve = FALSE) Extract rows that meet logical criteria.
mtcars > filter(mpg > 20)



filter_out(.data, ..., .preserve = FALSE) Drop rows that meet logical criteria (the complement to **filter()**). Missing values in the dropped condition are kept by default, so you don't need to guard against NA explicitly.
mtcars > filter_out(mpg > 20)



Use **when_any()** and **when_all()** inside **filter()** to combine multiple conditions.
mtcars > filter(when_any(mpg > 30, hp > 300))



distinct(.data, ..., .keep_all = FALSE) Remove rows with duplicate values.
mtcars > distinct(gear)



slice(.data, ..., .preserve = FALSE) Select rows by position.
mtcars > slice(10:15)



slice_sample(.data, ..., n, prop, weight_by = NULL, replace = FALSE) Randomly select rows. Use **n** to select a number of rows and **prop** to select a fraction of rows.
mtcars > slice_sample(n = 5, replace = TRUE)



slice_min(.data, order_by, ..., n, prop, with_ties = TRUE) and **slice_max()** Select rows with the lowest and highest values.
mtcars > slice_min(mpg, prop = 0.25)



slice_head(.data, ..., n, prop) and **slice_tail()** Select the first or last rows.
mtcars > slice_head(n = 5)

Logical and boolean operators to use with filter()

==	<	<=	is.na()	%in%		xor()
!=	>	>=	!is.na()	!	&	

See **?base::Logic** and **?Comparison** for help.

ARRANGE CASES



arrange(.data, ..., .by_group = FALSE) Order rows by values of a column or columns (low to high), use with **desc()** to order from high to low.
mtcars > arrange(mpg)
mtcars > arrange(desc(mpg))

ADD CASES



add_row(.data, ..., .before = NULL, .after = NULL) Add one or more rows to a table.
cars > add_row(speed = 1, dist = 1)

Manipulate Variables

EXTRACT VARIABLES

Column functions return a set of columns as a new vector or table.



pull(.data, var = -1, name = NULL, ...) Extract column values as a vector, by name or index.
mtcars > pull(wt)



select(.data, ...) Extract columns as a table.
mtcars > select(mpg, wt)



relocate(.data, ..., .before = NULL, .after = NULL) Move columns to new position.
mtcars > relocate(mpg, cyl, .after = last_col())

Use these helpers with select() and across()

e.g. mtcars > select(mpg:cyl)

contains(match)	num_range(prefix, range)	; e.g., mpg:cyl
ends_with(match)	all_of(x)/any_of(x, ..., vars)	!; e.g., !gear
starts_with(match)	matches(match)	everything()

MANIPULATE MULTIPLE VARIABLES AT ONCE

df <- tibble(x_1 = c(1, 2), x_2 = c(3, 4), y = c(4, 5))



across(.cols, .funs, ..., .names = NULL) Summarize or mutate multiple columns in the same way.
df > summarize(across(everything(), mean))



c_across(.cols) Compute across columns in row-wise data.
df > rowwise() > mutate(x_total = sum(c_across(1:2)))

MAKE NEW VARIABLES

Apply **vectorized functions** to columns. Vectorized functions take vectors as input and return vectors of the same length as output (see back).

vectorized function



mutate(.data, ..., .keep = "all", .before = NULL, .after = NULL) Compute new column(s). Also **add_column()**.
mtcars > mutate(gpm = 1 / mpg)
mtcars > mutate(gpm = 1 / mpg, .keep = "none")



rename(.data, ...) Rename columns. Use **rename_with()** to rename with a function.
mtcars > rename(miles_per_gallon = mpg)



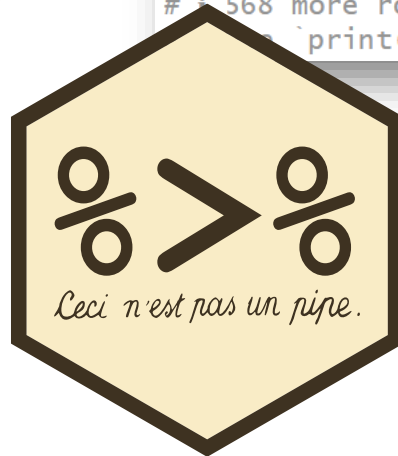
DATA PIPELINES

1.



```
> tibble::tibble(ChickWeight)
# A tibble: 578 x 4
  weight  Time Chick Diet
  <dbl> <dbl> <ord> <fct>
1     42     0  1     1
2     51     2  1     1
3     59     4  1     1
4     64     6  1     1
5     76     8  1     1
6     93    10  1     1
7    106    12  1     1
8    125    14  1     1
9    149    16  1     1
10   171    18  1     1
# ... 568 more rows
#> `print(n = ...)` to see more rows
```

2.



Basic piping

- `x %>% f` is equivalent to `f(x)`
- `x %>% f(y)` is equivalent to `f(x, y)`
- `x %>% f %>% g %>% h` is equivalent to `h(g(f(x)))`

3.



dplyr is a grammar of data manipulation, providing a consistent set of verbs that help you solve the most common data manipulation challenges:

- `mutate()` adds new variables that are functions of existing variables
- `select()` picks variables based on their names.
- `filter()` picks cases based on their values.
- `summarise()` reduces multiple values down to a single summary.
- `arrange()` changes the ordering of the rows.

USING DATA PIPELINES 1

- Save the ChickWeight data as a tibble in the variable CW.
- Using the Dplyr functions in “Group Cases” and “Summarize Cases” obtain the following information:
 - How many chicks are in each Diet group?
 - What is the average weight of the chicks in each group?
- Using the Dplyr functions in “Manipulative Cases” save the following data in the variables AA and BB, respectively:
 - Those chicks that are in group 4.
 - Those chicks that have been in the sample more than 18 days and that weigh more than 90 gms.

USING DATA PIPELINES 2

- Using the Dplyr functions in “Group Cases” and the Dplyr function “mutate” do the following:
 - Standardize the chicks’ weight to have mean 0 and SD of 1.
- Open the csv file “FarmInfo.csv” in R. Using the functions in the “Combine Tables” section add those values as new columns of CW and save it in the variable CW_Farm.
- Save the combined data as a new csv file called “CW_Farm.csv”.
- Are chicks in Organic farms on average thinner than those in Non-Organic farms?

5 MINS BREAK



5. DATA VISUALIZATION

❖ GGPLOT: [HTTPS://WWW.TIDYVERSE.ORG/](https://www.tidyverse.org/)



ggplot2 is a system for declaratively creating graphics, based on [The Grammar of Graphics](#). You provide the data, tell ggplot2 how to map variables to aesthetics, what graphical primitives to use, and it takes care of the details.

Source: <https://ggplot2.tidyverse.org/>



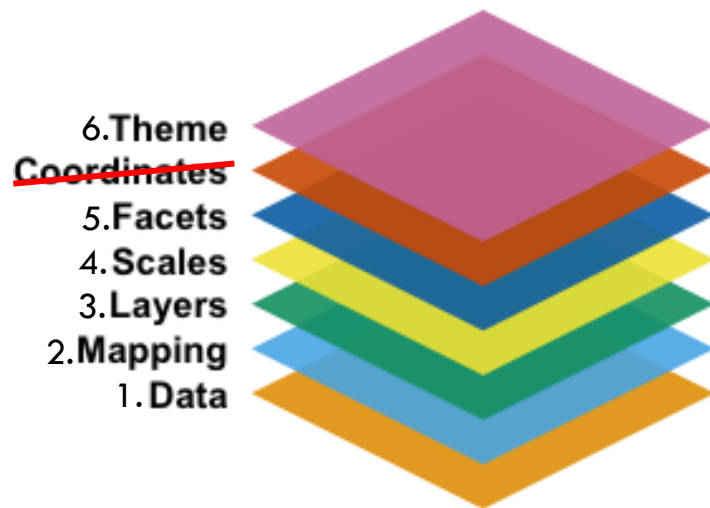
GGPLOT2

ggplot2 is a package for creating graphs, which follows the grammar of graphs.

What is the grammar of graphs?

The grammar of graphs tells us that "... Every statistical graph is a mapping of data to a set of geometric objects (points, lines, bars) that contain aesthetic attributes (color, shape, size). The graph can even have statistical transformations of the data, and these are drawn on a specific Cartesian plane."

THE COMPONENTS OF A GRAPH

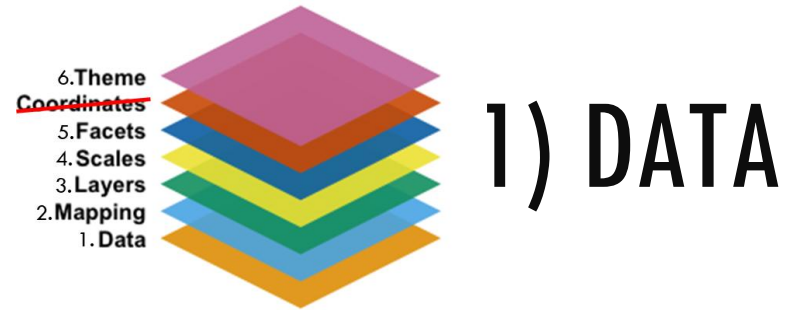


- 1) **Data** is the foundation of every graphic. The system works best if the **data is provided in a tidy format**, which briefly means a rectangular data frame structure where rows are observations and columns are variables.
- 2) The **mapping** of a plot is a set of instructions on how parts of the data are mapped onto aesthetic attributes of geometric objects. It is the 'dictionary' to translate tidy data to the graphics system.
- 3) **Layers** made of geometric objects (which we'll call **geoms**) and statistical transformations (which we'll call **stats**). Geoms represent what you see on the graph: points, lines, polygons, etc. Stats are a summary of the data being observed.
- 4) **Scales** maps the values of the data to values in an aesthetic space, such as color, size, or shape. Scales draws the legends or axes, which provides reverse mapping that makes it possible to read the original data from the graph.
- 5) The **facet** specification describes how to divide data into subsets and how to display it in subgraphs within the same graph. This is also called a conditioner.
- 6) A **theme** controls the points that are displayed, such as font size and color.

A quick guide is here: <https://github.com/rstudio/cheatsheets/blob/main/data-visualization.pdf>

FOR THE EXAMPLE, I WILL USE IRIS.

FOR THE EXERCISES, YOU WILL USE CW_FARMS.

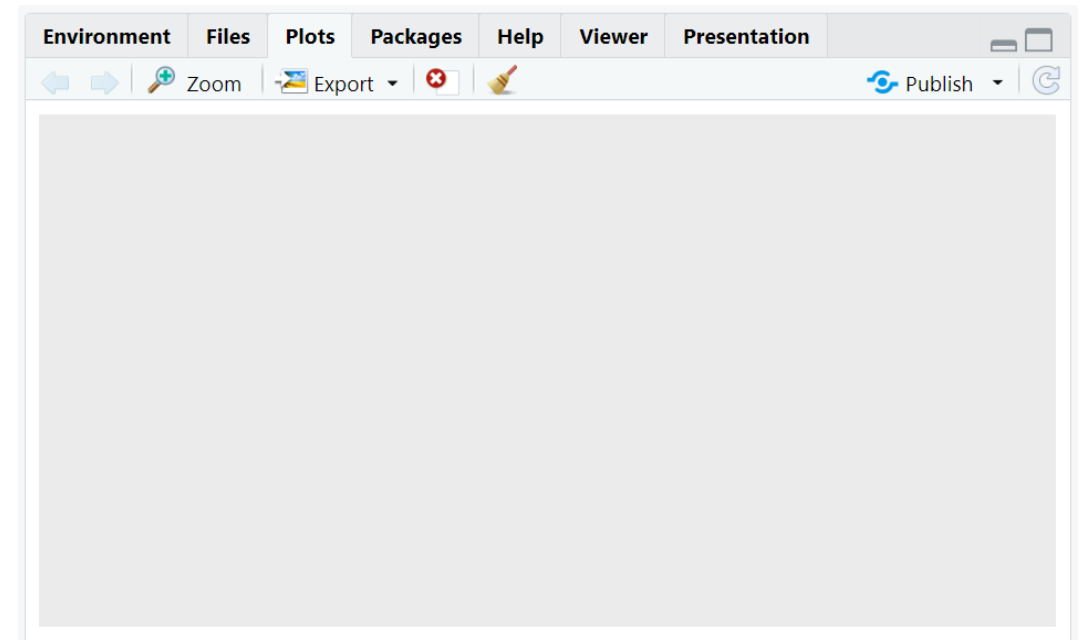


This is just the Data Frame that you want to visualize.

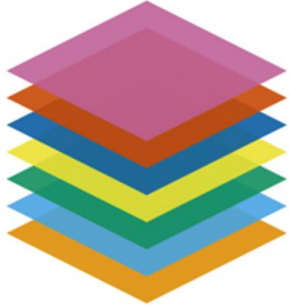
```
library(tidyverse)
IRIS<-tibble(iris)
```

1.Data

```
IRIS%>%
  ggplot()
```



6.Theme
~~Coordinates~~
5.Facets
4.Scales
3.Layers
2.Mapping
1.Data



2) MAPPING

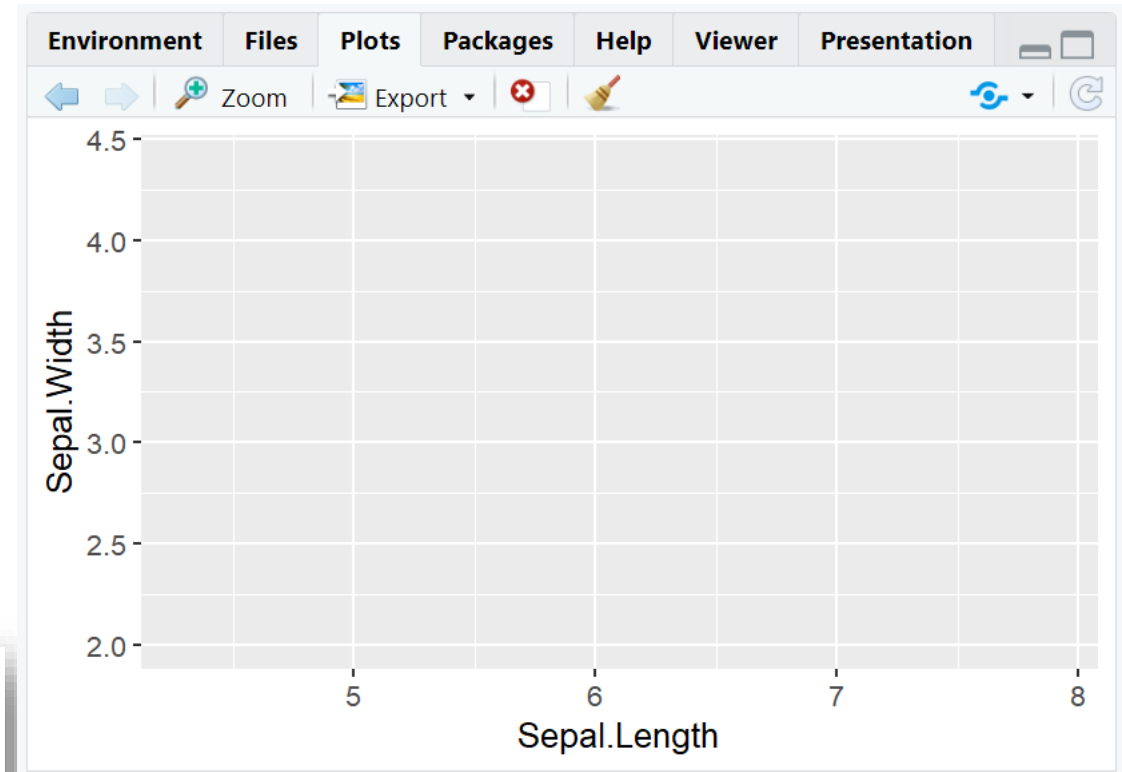
This refers to the variables you want to map: X and Y.

This is done by nesting the function `aes()` inside the function `ggplot()`.

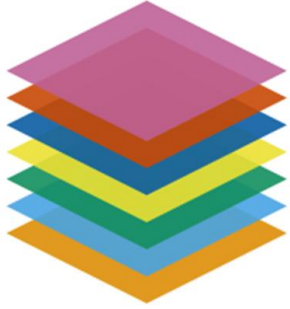
Inside the function `aes()`, you must add the names of the columns (X and/or Y) from your data that you want to plot.

2. Mapping: Variables in the Data

```
IRIS%>%  
ggplot(aes(x=Sepal.Length,y=Sepal.Width))
```



6.Theme
~~Coordinates~~
5.Facets
4.Scales
3.Layers
2.Mapping
1.Data



3) LAYER: GEOMS

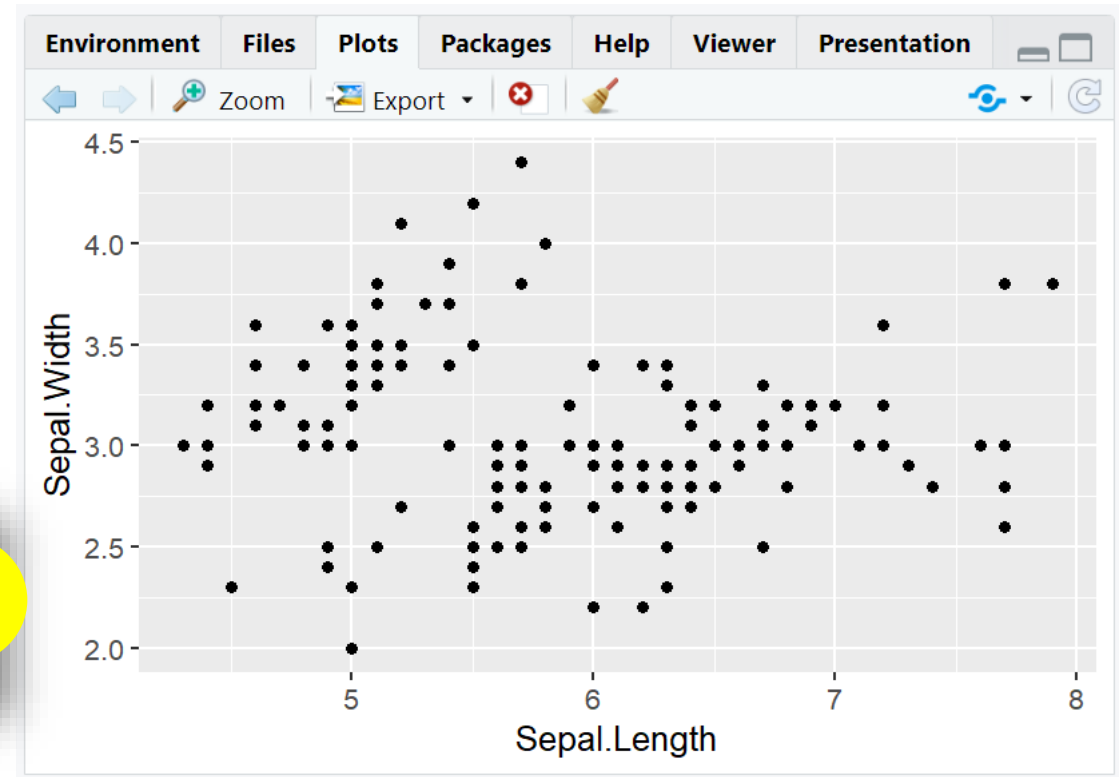
This refers to how the visualization between the variables (X and Y) will be done. You can visualize it as dots, or as bars, or as whatever you think fits.

In Rstudio, you can see all the geometries by typing “geom_”, followed by pressing tap on your keyboard.

```
IRIS%>%  
ggplot(aes(x=Sepal.Length,y=Sepal.Width))  
geom_point()
```

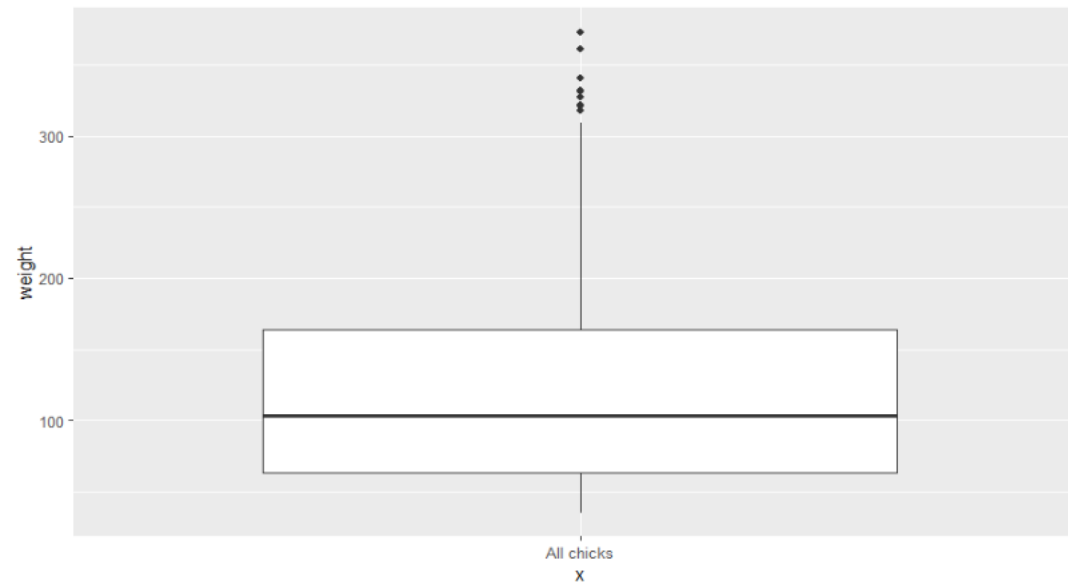
3. Scale:
Geometric Object

Add and
transform the
graph

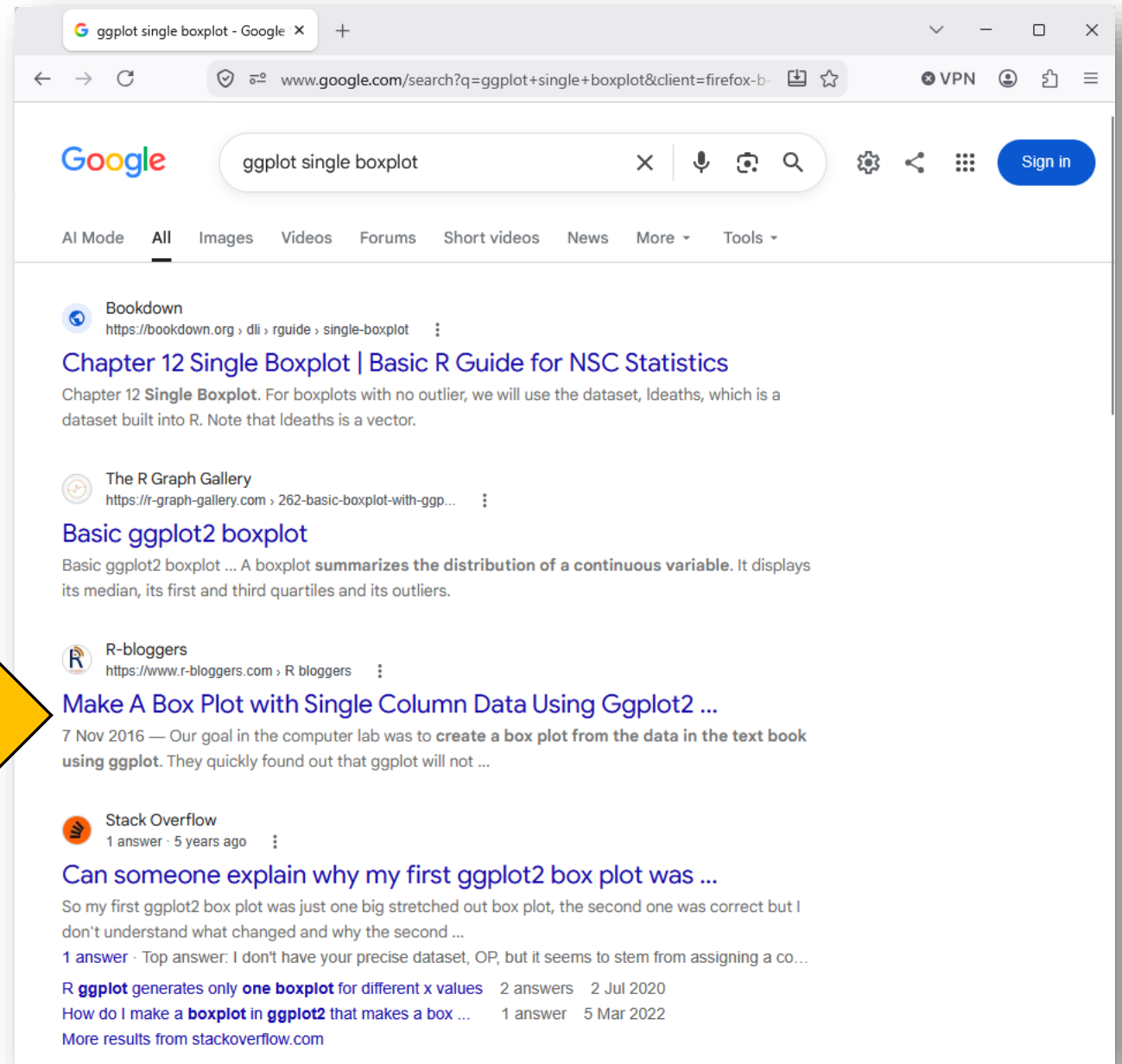
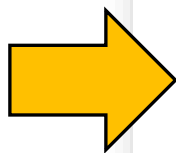
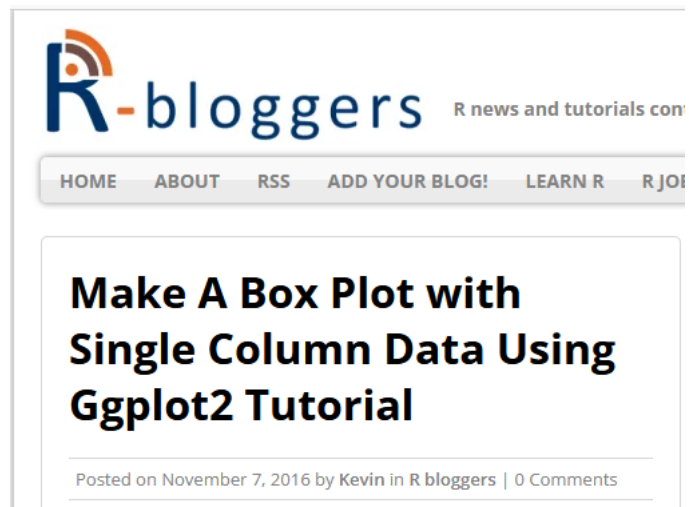


USING THE GRAMMAR OF GRAPHS

- Using the dataframe `CW_Farm`, how would you map the data and what geometry would you use to obtain...?



LOOKING FOR HELP 4: INTERNET FORUMS



Make A Box Plot with Single Column Data Using Ggplot2 Tutorial

Posted on November 7, 2016 by Kevin in R bloggers | 0 Comments

[This article was first published on [R – Saturn Science](#), and kindly contributed to [R-bloggers](#). (You can report issue about the content on this page [here](#))

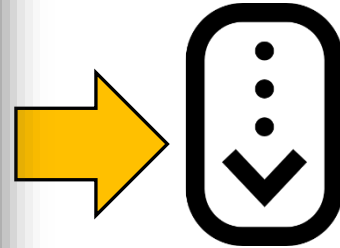
Want to share your content on R-bloggers? [click here](#) if you have a [blog](#), or [here](#) if you don't.

f Share

X Tweet

Last week I had my class practice making a box plot using the [data](#) on page 66 in *The Practice of Statistics* 4th Edition (TPS 4ed) text book. [Data visualization workshop](#)

I'm still going over the details of making a box plot with just a



Code for male data

```
library("dplyr", lib.loc="/Library/Frameworks/R.framework/Versions/3.3/Resource
library("ggplot2", lib.loc="/Library/Frameworks/R.framework/Versions/3.3/Resou

male = data.frame(c(127,44,28,83,0,6,78,6,5,213,73,20,214,28,11)) # data from
ggplot(data = male, aes(x = "", y = male)) +
  geom_boxplot() +
  coord_cartesian(ylim = c(0, 150)) # I set the y axis scale so the plot looks
```



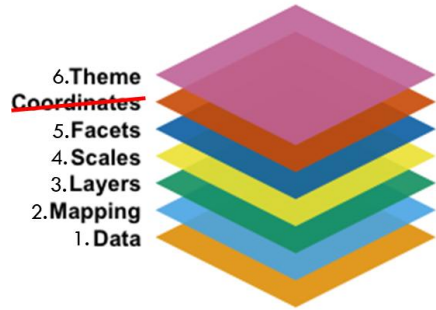
Copy-paste it in your R-script.

Modify it
until you
get what
you need!



```
male = data.frame(male=c(127,44,28,83,0,6,78,6,5,213,73,20,214,28,11))
ggplot(data = male, aes(x = "", y = male)) +
  geom_boxplot() +
  coord_cartesian(ylim = c(0, 150))
```

```
CW_Farms%>%
  ggplot(aes(y=weight,x="All chicks"))+
  geom_boxplot()
```



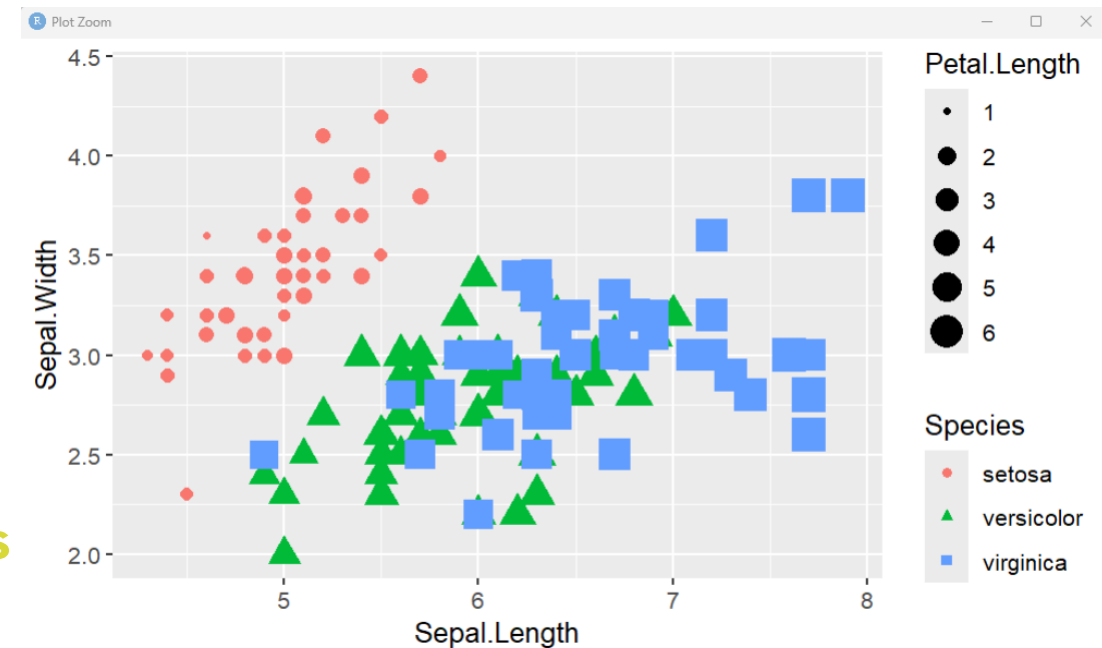
4) SCALES: COLOR, SIZE, SHAPE, AND OTHER AESTHETIC ATTRIBUTES

Let's say you chose to visualize the relation between your variables as dots. The scales refer to aesthetic changes to these dots. For example, you can choose to visualize the dots relative to a categorical variables (Z). So, instead of only black dots, you would have other shapes and colors depending on Z. However, their location would remain based on the value of X, Y.

```
IRIS%>%
```

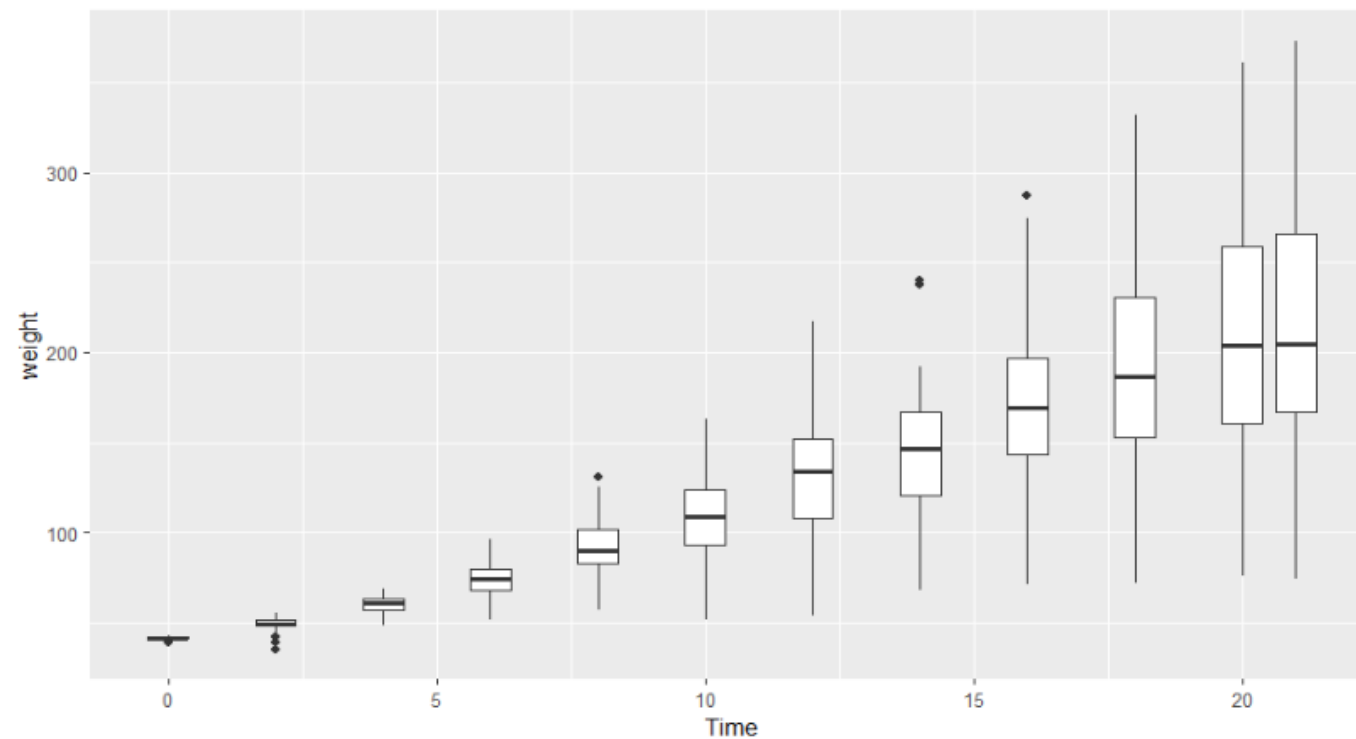
```
ggplot(aes(x=Sepal.Length, y=Sepal.Width,  
           color=Species,  
           shape=Species,  
           size=Petal.Length)) +  
geom_point()
```

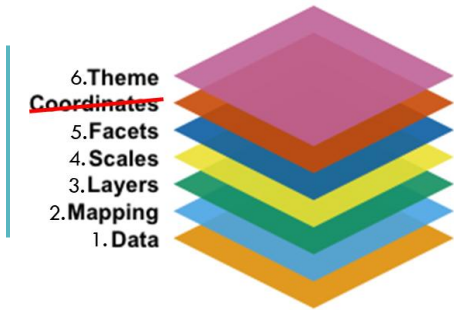
← 4. Scales



USING THE GRAMMAR OF GRAPHS

- What scale do you need to use to obtain...?



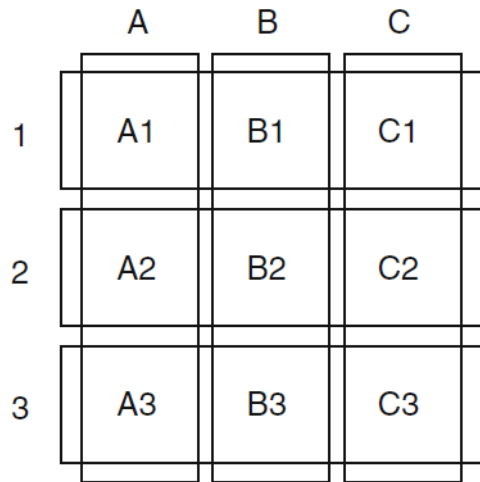


5) FACETS

ONE%>%

```
ggplot(aes(Species, Petal.Length)) +  
  geom_col(position = "dodge") +  
  facet_grid(DifMEAN~.)
```

Faceting refers to creating the same graph but splitting it by specific categorical variables. The functions are `facet_grid()`, and `facet_wrap()`. As parameter, you need to pass the variables that will be used to split the plots: **X1+X2~Y1+Y2**



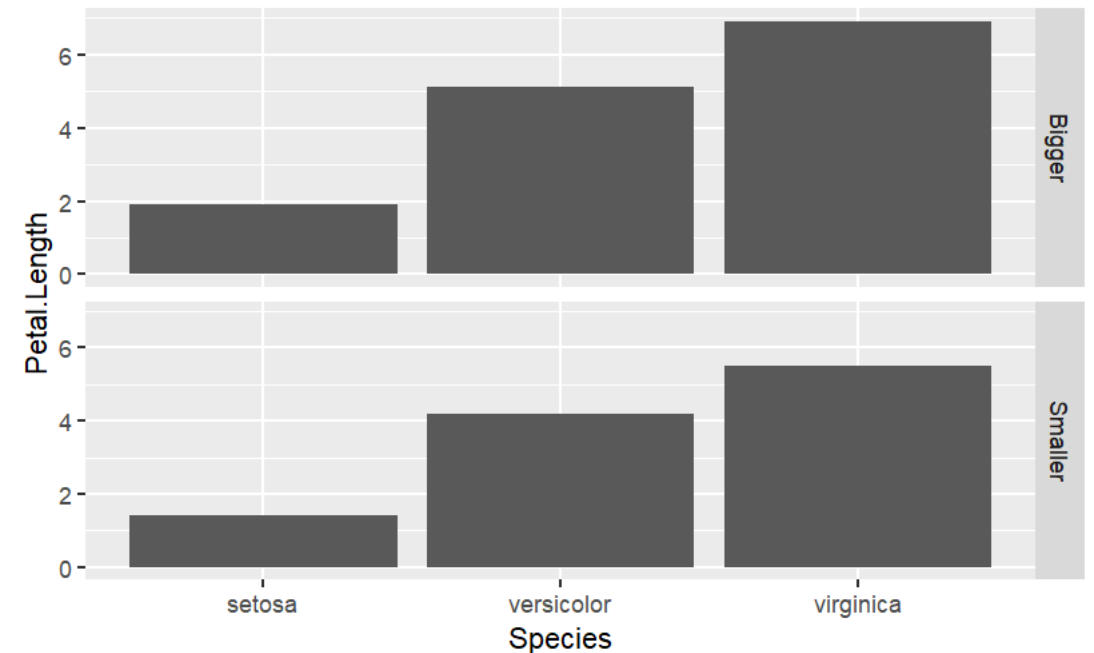
facet_grid

They share axes



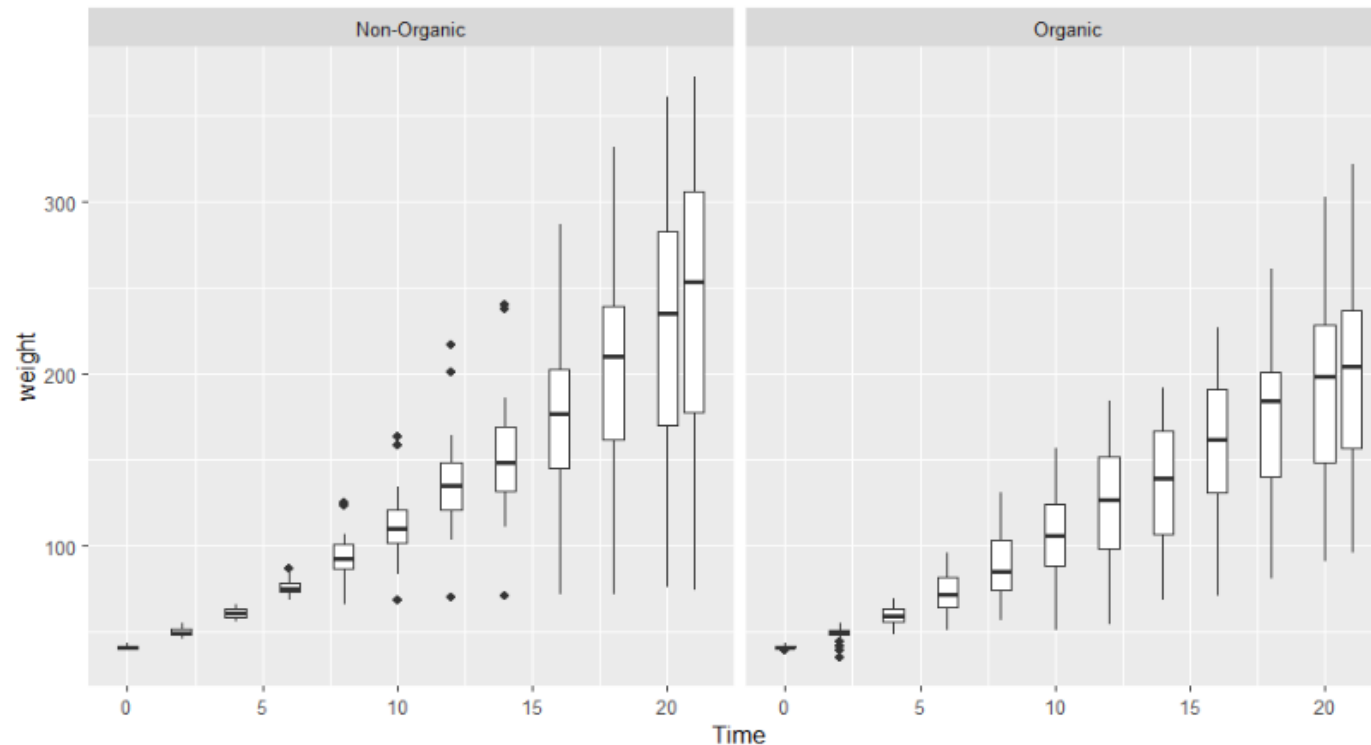
facet_wrap

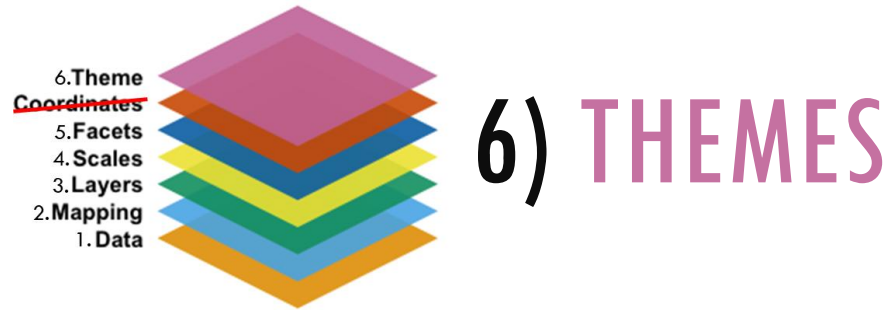
Each chart has its axes



USING THE GRAMMAR OF GRAPHS

- What facet function and formula do you need to pass to graph...?

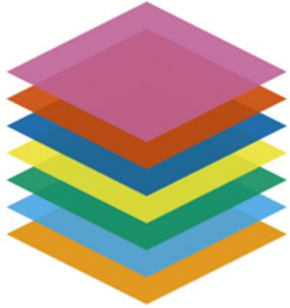




Themes allows you to exercise fine **control** over **the non-data elements of your plot**.

The theme system does not affect how the data is rendered by geoms, or how it is transformed by scales. Themes don't change the perceptual properties of the plot, but **they do help you make the plot aesthetically pleasing** or match an existing style guide. **Themes give you control over** things like **fonts, ticks, panel strips, and backgrounds**.

6.Theme
Coordinates
5.Facets
4.Scales
3.Layers
2.Mapping
1.Data

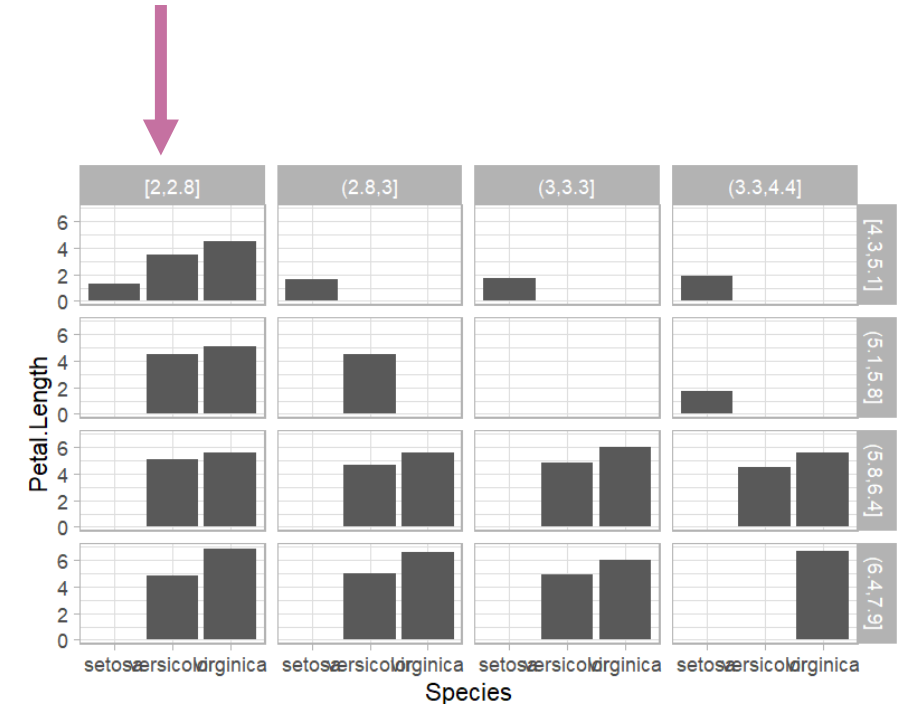


6) THEMES: PREDEFINED

EX_3.4%>%

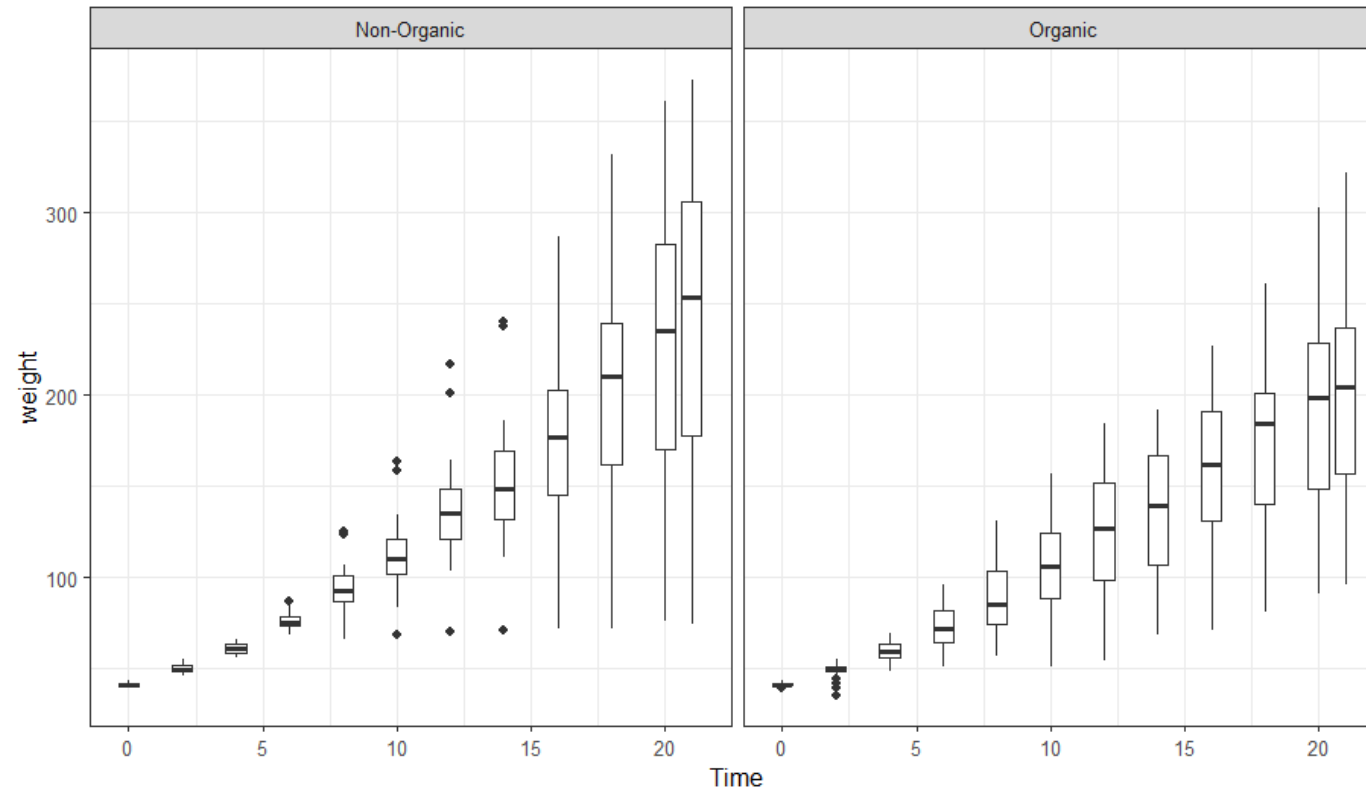
```
ggplot(aes(Species, Petal.Length)) +  
  geom_col(position = "dodge") +  
  facet_grid(Q_SL~Q_SW) +  
  theme_light()
```

- `theme_bw()`: a variation on `theme_grey()` that uses a white background and thin grey grid lines.
- `theme_linedraw()`: A theme with only black lines of various widths on white backgrounds, reminiscent of a line drawing.
- `theme_light()`: similar to `theme_linedraw()` but with light grey lines and axes, to direct more attention towards the data.
- `theme_dark()`: the dark cousin of `theme_light()`, with similar line sizes but a dark background. Useful to make thin coloured lines pop out.
- `theme_minimal()`: A minimalistic theme with no background annotations.
- `theme_classic()`: A classic-looking theme, with x and y axis lines and no gridlines.
- `theme_void()`: A completely empty theme.



USING THE GRAMMAR OF GRAPHS

- Which theme gives you...?



REFERENCES

- ❑ Gil-Clavel, Sofia. (2026). R-Course: R for Social Scientists (Version v.2) Zenodo.
<https://doi.org/10.5281/zenodo.20828330>
- ❑ Wickham, Hadley, Mine Çetinkaya-Rundel, and Garrett Grolemund. *R for data science*. " O'Reilly Media, Inc.", 2023. Accessed May 7, 2024.
<https://r4ds.hadley.nz/>.
- ❑ Wickham, Hadley. *Ggplot2*. Use R! Cham: Springer International Publishing, 2016.
<https://doi.org/10.1007/978-3-319-24277-4>.



Join us!



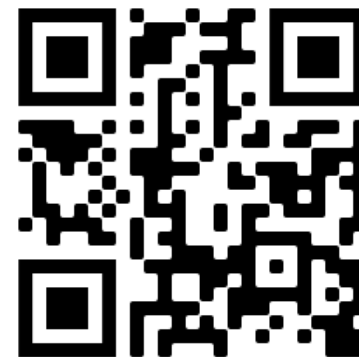
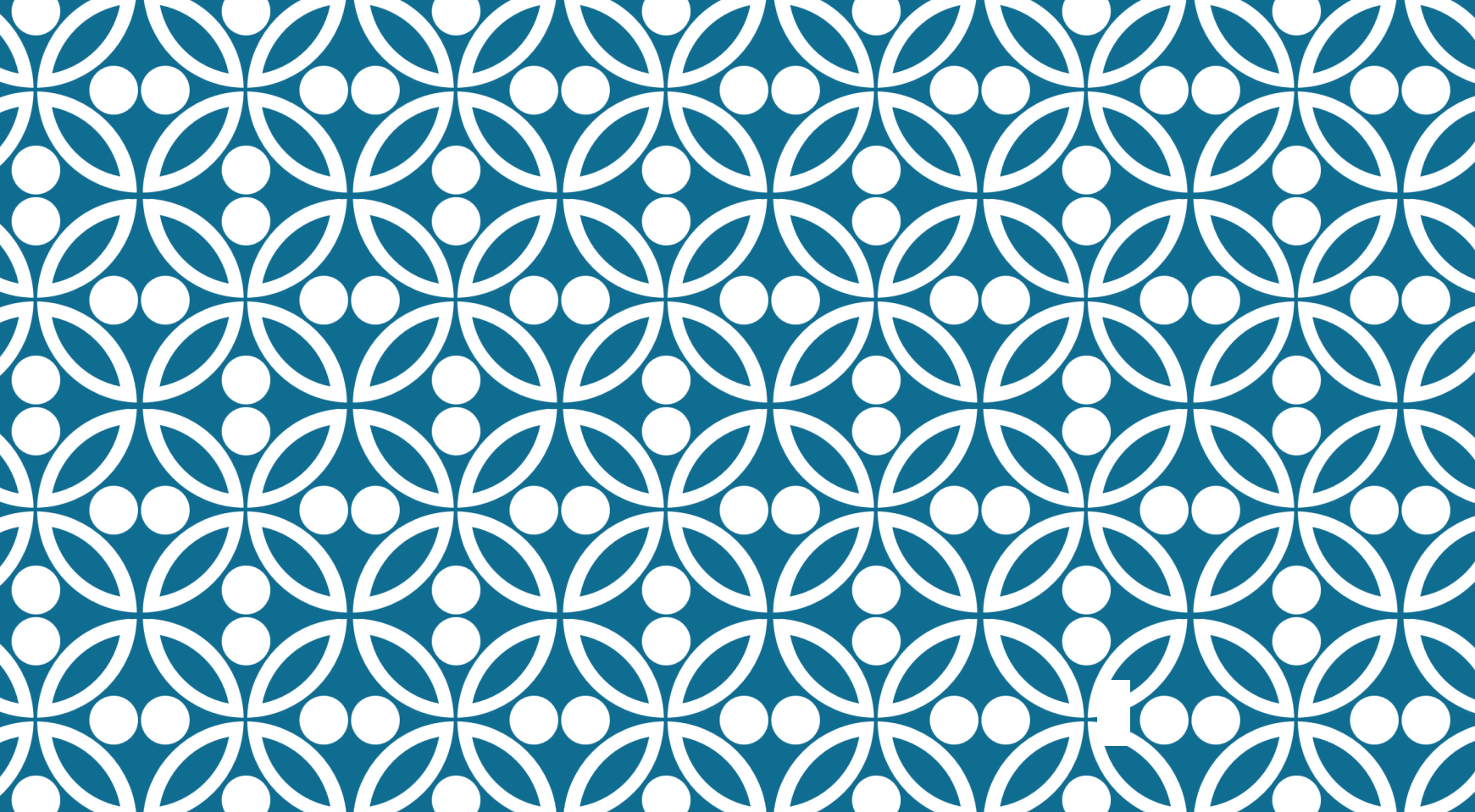
<https://forms.office.com/e/8Bgd2YsasJ>

All about the lab:

<https://societal-analytics.nl/>

Contact us at:

analytics-lab.fsw@vu.nl



<https://sofiagil.github.io/>

THANKS!

Dr. Sofia Gil-Clavel